



UNIVERSITY OF BUEA

College of Technology

Department of Computer Engineering

Real-Time Facial Recognition Deep Learning Architecture for Secure Campus Environments

A Proposal Submitted to the Department of Computer Engineering, College of Technology of
the University of Buea, in Partial Fulfillment of the Requirements for the Award of the Master
of Technology (M.Tech.) Degree in Software Engineering

BY

Ambanumben Norbel Apono-Nzume

CT24P011, B Tech. Computer Engineering

Supervisor:

Dr. Tchapgha Tchito Christian

Lecturer

March 2026

Contents

List of Abbreviations	iv
List of Figures	vi
List of Tables	vii
List of Appendices	viii
1 Introduction	1
1.1 Background to the Study	1
1.2 Problem Statement	2
1.3 Rationale of the Study	2
1.4 Research Questions	4
1.5 Objectives and Hypothesis	4
1.6 Significance of the Study	5
2 Review of the Literature	7
2.1 Historical Overview of the Theory and Research Literature	7
2.2 Theory and Research Literature Specific to the Topic	8
2.2.1 Edge Computing for Computer Vision	8
2.2.2 Evolution of YOLO-Based Architectures	8
2.2.3 YOLO Optimization for Edge Deployment	9
2.2.4 Face Recognition Pipelines	11
2.2.5 Anomaly Detection Models	12
2.2.6 Robustness Techniques	12
2.2.7 Synthetic Data and Privacy	12
2.3 Critique of Current Literature and Summary of Known Results	12
2.4 Contribution This Study Will Make to the Literature	15
3 Materials and Methods	17
3.1 Research Design	17
3.2 Specific Procedures	17
3.2.1 Image Acquisition	17
3.2.2 Model Selection and Architecture Modification	18
3.2.3 Optimization	19

3.2.4	Mathematical Background	19
3.2.5	Detection-to-Recognition Pipeline	26
3.3	Testing and Evaluation	27
3.3.1	Ablation Study	27
3.3.2	Resolution Sensitivity	27
3.3.3	Environmental Stress Testing	27
3.3.4	Performance Metrics	28
3.4	Research Population or Sample	28
3.4.1	Training Data	28
3.4.2	Privacy-Preserving Training	28
3.4.3	Testing Data	28
3.5	Instrumentation (Pilot Study, Data Collection)	29
3.5.1	Hardware	29
3.5.2	Software	29
3.5.3	Computational Budget and Resource Requirements	29
3.5.4	Pilot Study	32
3.6	Treatment and Analysis of the Data	32
3.6.1	Statistical Analysis	34
3.7	Ethical Considerations	36
3.8	Scope of the Current Research	36
A	Supplementary Data	42
A.1	Complete Training Hyperparameters	42
A.2	Knowledge Distillation Hyperparameters	42
A.3	Dataset Statistics	42
A.4	Modified YOLO Architecture Specification	42
A.4.1	Backbone (MobileNetV3-Small)	44
A.4.2	Head (FPN/PAN Neck + Detection)	44
A.5	Quantization Calibration Configuration	44
A.6	Benchmark Protocol Details	46
A.7	Resolution Sweep Configuration	46
A.8	Environmental Stress Test Conditions	46
B	Research Instruments	48
B.1	Informed Consent Form	48
B.2	Volunteer Enrollment Protocol	49
B.3	Pilot Study Data Collection Protocol	50
B.3.1	Pre-Test Setup	51
B.3.2	Test Execution	51

B.3.3	Post-Test Analysis	52
B.4	System Evaluation Checklist	52

List of Abbreviations

AI	Artificial Intelligence
AMP	Automatic Mixed Precision
AP	Average Precision
ArcFace	Additive Angular Margin Loss (face recognition method)
ARM	Advanced RISC Machine (processor architecture)
C3k2	Cross-Stage Partial module variant (YOLO architectural block)
CBAM	Convolutional Block Attention Module
CCTV	Closed-Circuit Television
CIoU	Complete Intersection over Union
CLAHE	Contrast-Limited Adaptive Histogram Equalization
CNN	Convolutional Neural Network
CPU	Central Processing Unit
CUDA	Compute Unified Device Architecture (NVIDIA)
DL	Deep Learning
DnCNN	Denoising Convolutional Neural Network
EER	Equal Error Rate
FAR	False Acceptance Rate
FLOPs	Floating Point Operations
FP16	16-bit Floating Point (half precision)
FP32	32-bit Floating Point (single precision)
FPS	Frames Per Second
FRR	False Rejection Rate
GAN	Generative Adversarial Network
GB	Gigabyte
GAP	Global Average Pooling
GPU	Graphics Processing Unit
HOG	Histogram of Oriented Gradients
iGPU	Integrated Graphics Processing Unit

INT8	8-bit Integer (quantization precision)
IVS	Intelligent Video Surveillance
LFW	Labeled Faces in the Wild (benchmark dataset)
LSTM	Long Short-Term Memory
KD	Knowledge Distillation
KL	Kullback–Leibler (divergence)
mAP	mean Average Precision
MB	Megabyte
MLP	Multi-Layer Perceptron
MSE	Mean Squared Error
MTCNN	Multi-Task Cascaded Convolutional Networks
NCNN	Neural Network inference framework (Tencent)
NNCF	Neural Network Compression Framework (Intel OpenVINO)
NWD	Normalized Wasserstein Distance
OpenVINO	Open Visual Inference and Neural Network Optimization (Intel)
PTQ	Post-Training Quantization
QDQ	Quantize-DeQuantize
RAM	Random Access Memory
RFE	Receptive Field Enhancement
ROC	Receiver Operating Characteristic
RPi	Raspberry Pi
RTSP	Real Time Streaming Protocol
SE	Squeeze-and-Excitation (block)
SEAM	Scale-Equalizing Anchor Matching
SoC	System-on-Chip
TDP	Thermal Design Power
TensorRT	Tensor Runtime (NVIDIA inference optimization library)
VNNI	Vector Neural Network Instructions (Intel)
WIDER FACE	Wider Face detection benchmark dataset
YOLO	You Only Look Once

List of Figures

1.1	Proposed edge-cloud hierarchical architecture for campus face recognition.	3
2.1	Historical evolution of face detection and recognition methods.	8
2.2	Evolution of YOLO architectures for face detection.	10
2.3	Literature gap analysis matrix mapping existing works to identified re- search gaps.	14
3.1	Proposed modified YOLO architecture for face detection.	18
3.2	Four-phase model optimization strategy.	20
3.3	Internal architecture of the Convolutional Block Attention Module (CBAM) [20].	22
3.4	Knowledge distillation architecture showing teacher–student training. . .	25
3.5	End-to-end detection-to-recognition inference pipeline.	27
3.6	Cross-platform deployment and evaluation workflow.	33
3.7	Complete experimental protocol spanning training, optimization, bench- marking, and evaluation.	35

List of Tables

2.1	Capability comparison of this study vs. prior work across six key dimensions.	14
2.2	Comparison of Foundational Works in YOLO-Based Face Detection and Edge Deployment	16
3.1	Target Edge Hardware Specifications	29
3.2	Training Compute Requirements per Experimental Phase	30
3.3	GPU Options for Training Workload	30
3.4	Edge Hardware Acquisition Budget	31
3.5	Total Research Budget Summary	31
A.1	Complete Training Hyperparameters	43
A.2	Knowledge Distillation Hyperparameters	43
A.3	Dataset Statistics	44
A.4	Modified Backbone Architecture (MobileNetV3-Small)	44
A.5	Modified Head Architecture (FPN/PAN Neck with CBAM and SEAM) .	45
A.6	INT8 Quantization Calibration Configuration per Platform	45
A.7	Resolution Sweep Parameters	46
A.8	Environmental Stress Test Dataset Conditions	47
B.1	Pre-Experiment Verification Checklist	52

List of Appendices

Appendix A: Supplementary Data (Hyperparameters, Dataset Statistics, Architecture Specification, Benchmark Protocol)

Appendix B: Research Instruments (Consent Form, Enrollment Protocol, Pilot Study Protocol, Evaluation Checklist)

Chapter 1

Introduction

1.1 Background to the Study

In the modern educational sector, ensuring the security of students and staff across busy university campuses is paramount. Traditional security measures, such as manual guarding or passive Closed-Circuit Television (CCTV), are increasingly viewed as insufficient due to human error, high costs, and the inability to process vast amounts of video data in real-time [1, 2]. In Sub-Saharan Africa, university campuses face additional constraints: limited security budgets, unreliable internet connectivity for cloud-based solutions, and frequent power fluctuations that demand energy-efficient hardware. Consequently, there has been a significant shift towards Intelligent Video Surveillance (IVS) systems that utilize computer vision to automatically detect and recognize objects and humans without constant human supervision [3].

Facial recognition technology has emerged as a powerful tool for identity verification [2]. Early approaches relied on handcrafted features such as Eigenfaces [4] and Histogram of Oriented Gradients (HOG) descriptors [5], followed by cascade detectors like Viola–Jones [6] and hybrid traditional-ML methods combining PCA with Hidden Markov Models [7]. The deep learning revolution, catalyzed by AlexNet’s success on ImageNet [8], enabled end-to-end learned representations that dramatically improved both detection and recognition accuracy. However, conventional deep learning pipelines often separate face detection from recognition (e.g., MTCNN [9] followed by FaceNet [10]), which increases computational complexity and latency [11].

Recent advancements in the You Only Look Once (YOLO) architecture [12] offer a unified, single-stage approach that balances speed and accuracy, making it highly suitable for dynamic, real-world environments [11]. Domain-specific modifications such as YOLO-FaceV2 [13] have demonstrated that YOLO can be adapted for face-specific tasks with attention mechanisms, occlusion handling, and specialized loss functions. When combined with lightweight backbone architectures such as MobileNetV3 [14], these detectors become viable candidates for deployment on resource-constrained edge hardware.

1.2 Problem Statement

A fundamental “deployment contradiction” exists in implementing advanced AI for campus security: the computational demands of state-of-the-art Deep Learning (DL) models often exceed the resource capabilities of edge devices (e.g., surveillance cameras, local gateways) [15]. While traditional multi-stage models achieve high accuracy, they lag significantly in inference speed compared to modern single-stage detectors like YOLO, rendering them less effective for real-time anomaly detection [16]. Furthermore, standard YOLO architectures may still be too heavy for constrained devices or struggle with specific challenges such as high-overhead angles and occlusion in crowded locations on campus [17, 18]. Relying solely on the cloud for processing introduces unacceptable latency and privacy risks regarding student biometric data [15].

This challenge is particularly acute for universities in Sub-Saharan Africa, where cloud infrastructure is often limited by low bandwidth and high latency, making real-time cloud-based facial recognition impractical. Edge-based processing offers a path forward, but no existing work evaluates face-specific YOLO architectural modifications across multiple affordable edge platforms under identical conditions.

1.3 Rationale of the Study

This study proposes an edge-centric computing framework to resolve the latency and privacy bottlenecks of cloud-centric surveillance. By processing data at the network periphery (the “edge”), using lightweight “nano” models (e.g., YOLOv11n, YOLOv12n), the system can achieve ultra-low latency compared to traditional methods and reduce upstream bandwidth usage by transmitting only extracted features or alerts rather than raw video streams [15, 19]. This approach is critical for delay-sensitive applications like real-time threat detection and anomaly alerting [3].

Furthermore, processing biometric data locally enhances data locality and privacy compliance—raw facial images never leave the edge device, and only compact 512-dimensional embeddings or anonymized alerts are transmitted to the cloud layer [15]. Integrating architectural improvements like the Convolutional Block Attention Module (CBAM) [20] ensures the system remains robust even under uncontrolled campus conditions such as low lighting, partial occlusion, and crowded entry points [17].

Figure 1.1 illustrates the proposed three-tier architecture.

The architecture follows the three-tier hierarchy defined by Wang et al. [15] and the edge computing paradigm of Shi et al. [19]. The Device Layer captures video via IP cameras at campus entry points and high-traffic areas. The Edge Layer performs real-time YOLO detection, five-point landmark alignment, and ArcFace [21] embedding extraction, matching against a locally cached enrollment database. The Cloud Layer

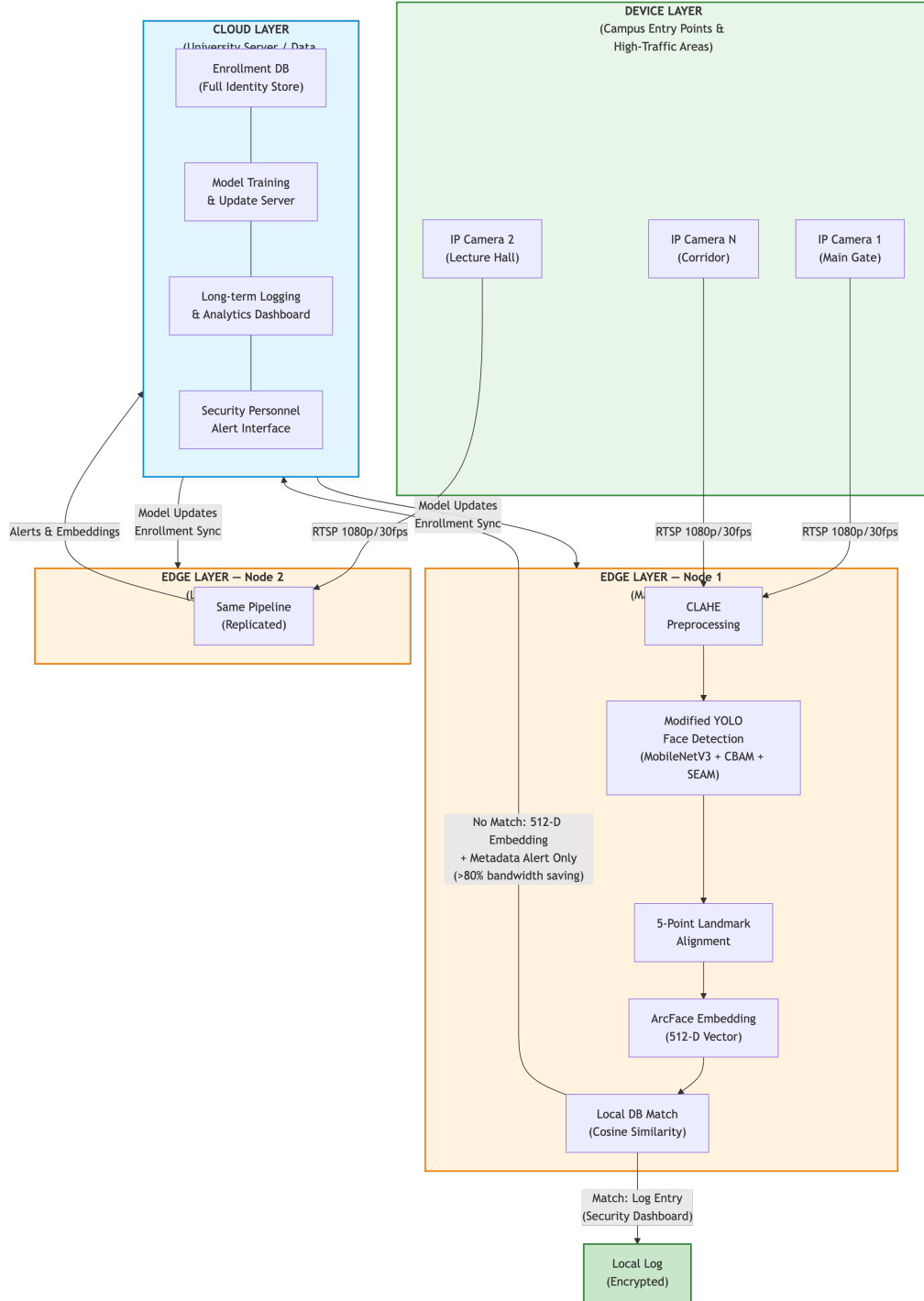


Figure 1.1: Proposed edge-cloud hierarchical architecture for campus face recognition.

handles enrollment management, long-term logging, and model updates. Only compact 512-dimensional embeddings and metadata are transmitted upstream, achieving greater than 80% bandwidth reduction compared to streaming raw video. Unrecognized faces are flagged as high-risk and escalated as alerts to security personnel rather than autonomously controlling physical access.

1.4 Research Questions

This study seeks to answer the following research questions:

1. To what extent do face-specific architectural modifications (MobileNetV3 backbone, CBAM attention, SEAM module, NWD loss) improve YOLO detection accuracy on the WIDER FACE benchmark [22] while reducing model parameters?
2. How does INT8 post-training quantization affect detection accuracy (mAP) and inference latency across GPU-accelerated (Jetson), CPU+iGPU (Intel N100), and ARM CPU (Raspberry Pi 5) edge platforms?
3. Can an end-to-end detection-to-recognition pipeline (YOLO + ArcFace [21]) achieve real-time performance (<30 ms on GPU, <100 ms on CPU) while maintaining recognition accuracy above 95% on a campus test set?
4. What are the Pareto-optimal accuracy–latency–cost trade-offs across the three target edge platforms for campus deployment?

1.5 Objectives and Hypothesis

The primary objective is to design, optimize, and empirically validate a YOLO-based facial recognition pipeline for real-time campus security, benchmarked across three distinct edge hardware platforms.

Specific objectives are:

1. **Optimize YOLO Architectures for Face Detection on Edge Devices:** To adapt YOLOv11n/v12n architectures for face-specific detection through targeted modifications, including replacing the standard backbone with **MobileNetV3** [14] for parameter reduction, integrating **CBAM** attention modules [20] for enhanced feature selection under occlusion [17], and incorporating face-specific enhancements inspired by YOLO-FaceV2 (e.g., SEAM attention, NWD loss) [13]. The optimized models will be further compressed via **INT8 post-training quantization** [23] and evaluated for knowledge distillation from heavier variants [15].

2. **Benchmark Across Three Edge Platforms:** To conduct the first comprehensive cross-platform evaluation of optimized YOLO face detection on affordable edge hardware: **NVIDIA Jetson Orin Nano Super** (GPU-accelerated, TensorRT), **Intel N100 mini PC** (CPU+iGPU, OpenVINO), and **Raspberry Pi 5** (ARM CPU, NCNN [24]). Each platform will be evaluated on inference latency, throughput (FPS), accuracy (mAP@0.5), energy efficiency (Joules/query), and total system cost [25, 26].
3. **Implement and Validate a Complete Detection-to-Recognition Pipeline:** To deploy an end-to-end pipeline—YOLO detection, face alignment, ArcFace [21] embedding extraction, and database matching—operating in a hierarchical edge-cloud architecture [27]. The edge layer handles real-time detection and matching against an enrolled database of known individuals; the cloud layer manages enrollment, long-term logging, and model updates. The system will be stress-tested across input resolution scales (160×160 to 640×640) [16], crowd density scenarios [18], and environmental stressors (low light via CLAHE preprocessing [28], occlusion from masks/glasses).

Hypothesis: An architecturally optimized YOLO face detection model (with MobileNetV3 backbone, CBAM attention, and INT8 quantization) deployed on an edge-cloud hierarchy will achieve less than 30 ms detection latency on GPU-accelerated edge devices (Jetson) and less than 100 ms on CPU-only platforms (N100, Raspberry Pi 5), while maintaining recognition accuracy above 95% and reducing cloud bandwidth consumption by at least 80% compared to a cloud-only architecture. These latency targets are derived from existing benchmarks: YOLOv11n standalone achieves approximately 38 ms inference on Jetson AGX Orin hardware [23], suggesting that architectural optimization and INT8 quantization of the detector itself can push latency below 30 ms on newer Jetson Orin Nano Super hardware. The 100 ms threshold represents the upper bound for acceptable interactive response in real-time surveillance alert systems [25].

1.6 Significance of the Study

This study makes three contributions of practical and scholarly significance:

1. **Technical:** It provides the first empirical evaluation of YOLO-FaceV2-inspired architectural modifications (SEAM, NWD loss, MobileNetV3 backbone) applied to YOLOv11n/v12n, combined with INT8 quantization—a configuration untested in the literature.
2. **Comparative:** It introduces the first three-way cross-platform benchmark of YOLO face detection on affordable edge hardware (Jetson Orin Nano Super, Intel

N100, Raspberry Pi 5), filling a gap where existing works evaluate only a single platform.

3. **Practical:** The resulting framework targets deployment in resource-constrained campus environments across Sub-Saharan Africa, where cloud infrastructure is limited and security budgets favor affordable, locally-processed solutions.

Chapter 2

Review of the Literature

2.1 Historical Overview of the Theory and Research Literature

Anomaly detection and face recognition have evolved through distinct phases. Early methods relied on **handcrafted features** such as Histogram of Oriented Gradients (HOG) [5] and the Viola–Jones cascade detector [6], which were computationally light but struggled with complex environments [3, 29]. Geometric approaches such as Eigenfaces [4] represented faces as points in a low-dimensional subspace but were sensitive to illumination and pose variation. Aggarwal et al. extended these classical methods by combining Principal Component Analysis (PCA) with Hidden Markov Models and Artificial Neural Networks, demonstrating that hybrid traditional-ML pipelines could partially address pose and expression variability, though they remained fundamentally limited compared to deep representations [7]. The introduction of **Convolutional Neural Networks (CNNs)**, catalyzed by AlexNet’s ImageNet success in 2012 [8], marked a turning point, allowing systems to automatically learn spatiotemporal features from video data [3]. More recently, the field has moved toward **Generative Adversarial Networks (GANs)** and **Long Short-Term Memory (LSTM)** networks to model temporal continuity and detect behavioral anomalies (e.g., unauthorized access patterns) [3]. Similarly, face recognition transitioned from geometric methods to deep learning-based embedding approaches like **FaceNet** [10], which maps faces directly to a compact Euclidean space using triplet loss, and **ArcFace** [21], which employs an additive angular margin loss for superior discriminability. Figure 2.1 illustrates the major phases of this evolution and the key contributions at each stage.

As shown in Figure 2.1, the field progressed from handcrafted features (Eigenfaces [4], Viola–Jones [6], HOG [5]) through the deep learning revolution (AlexNet [8], MTCNN [9], FaceNet [10], ArcFace [21]) to edge-optimized architectures (MobileNetV3 [14], YOLO-FaceV2 [13], EdgeFace [30]). Each era addressed the limitations of its predecessor: handcrafted features were replaced by learned representations, which in turn required lightweight variants for deployment on resource-constrained hardware. This study unifies detection and recognition in a cross-platform edge deployment framework.

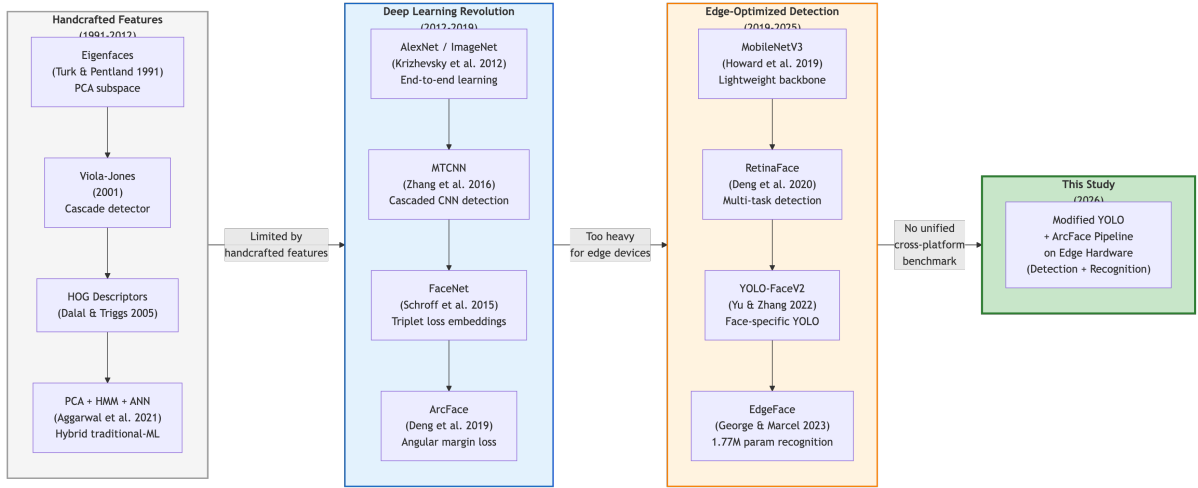


Figure 2.1: Historical evolution of face detection and recognition methods.

2.2 Theory and Research Literature Specific to the Topic

2.2.1 Edge Computing for Computer Vision

Edge computing represents a paradigm shift from centralized cloud processing to distributed inference at the network periphery [19]. Cognitive Edge Computing extends this concept by deploying advanced AI systems that reason and make decisions directly on edge devices [15]. The literature defines a three-tier hierarchy: Cloud (training/storage), Edge (inference/reasoning), and Device (sensing). Wang et al. emphasize that deploying reasoning-capable models at the edge requires cross-layer optimization, including model compression and system orchestration [15]. Recent works utilize collaborative inference, where lightweight models operate at the edge for immediate tasks, while complex reasoning is offloaded to the cloud only when necessary [15]. This paradigm is particularly relevant for campus environments in resource-constrained regions, where bandwidth limitations make cloud-only processing impractical.

2.2.2 Evolution of YOLO-Based Architectures

The YOLO family of detectors, introduced by Redmon et al. [12], established the single-stage detection paradigm that eliminates the need for separate region proposal and classification stages. Research has progressively moved away from multi-stage pipelines (e.g., MTCNN [9] + FaceNet [10]) toward these unified detectors:

- **YOLOv11 and YOLOv12:** Aymon et al. highlight the efficacy of “nano” versions of these models for facial tasks. YOLOv11 utilizes C3k2 modules for improved spatial encoding in noisy environments, while YOLOv12 integrates attention mechanisms for higher recall [11]. Both significantly outperform MTCNN in real-time

inference speeds [16]. The Ultralytics framework [31] provides a unified training and export pipeline for both architectures.

- **YOLOv8:** Yisihak and Li demonstrated the integration of YOLOv8 with anchor-free detection heads into embedded AI modules, achieving high precision for security gates [32].
- **Architectural Customization:** To handle high overhead angles typical of campus CCTV, Tang replaced the YOLOv5 backbone with **MobileNetV3** [14] and added **CBAM** attention modules [20], reducing parameters from 7.1M to 3.6M while increasing detection speed from 233 to 267 FPS. Combined with targeted data augmentation for overhead-angle faces, the fully improved algorithm achieved 93.01% mAP@0.5:0.95, outperforming comparison models by 2–3% [17].
- **YOLO-FaceV2:** Yu and Zhang introduced face-specific modifications to the YOLO architecture, including a **Receptive Field Enhancement (RFE)** module, **SEAM** attention for occlusion handling, and **Normalized Wasserstein Distance (NWD)** loss [33] for small-face detection. These modifications achieved state-of-the-art results on the WIDER FACE benchmark [22], demonstrating that domain-specific YOLO customization significantly outperforms generic object detectors for face tasks [13].

Figure 2.2 traces the evolution of YOLO architectures for face detection, highlighting the contributions of each referenced work and how this study synthesizes prior advances.

Figure 2.2 traces the progression from the original single-stage paradigm [12] through face-specific adaptations—YOLO-FaceV2 with SEAM, NWD, and RFE modules [13], and MobileNetV3+CBAM backbone replacement on YOLOv5 [17]—to recent edge optimization efforts including INT8 quantization on Jetson hardware [23] and YOLOv11/v12 baselines for facial tasks [11]. This study synthesizes contributions from each generation into a unified framework that combines all four architectural modifications with INT8 quantization and knowledge distillation, evaluated across three edge platforms.

2.2.3 YOLO Optimization for Edge Deployment

A critical and emerging subfield addresses the gap between YOLO’s detection accuracy and the computational constraints of edge hardware. Recent works have begun to bridge this divide:

- **Quantized YOLO for Masked Face Detection:** Choi et al. applied **DnCNN denoising** as a preprocessing stage with **QDQ INT8 quantization applied to the DnCNN module**, while YOLOv11 ran in FP32 precision, on an NVIDIA Jetson AGX Orin. The INT8-quantized DnCNN achieved near-FP16 accuracy (less

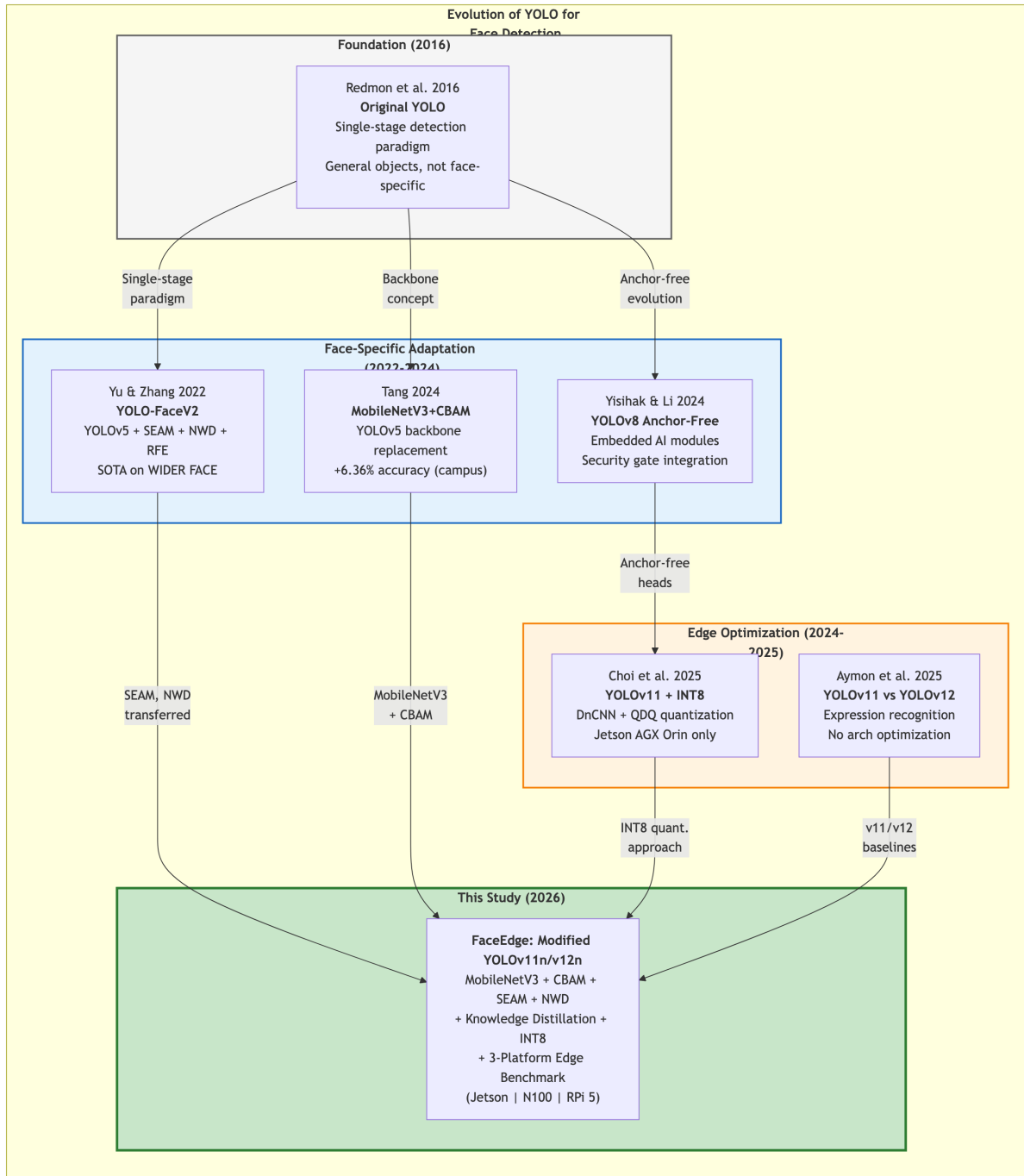


Figure 2.2: Evolution of YOLO architectures for face detection.

than 1% mAP degradation) while improving throughput by 31.6% (from 5.82 to 7.66 FPS for the combined pipeline) for masked face detection in surveillance scenarios [23].

- **Complete Detection-to-Recognition Pipelines on Embedded SoC:** Ozen et al. deployed a full **YOLO detection** → **landmark localization** → **face quality assessment** → **ArcFace recognition** pipeline on an embedded System-on-Chip, establishing that end-to-end face recognition is feasible on edge hardware when each pipeline stage is independently optimized [27].
- **Multi-Camera Edge Recognition:** Dang et al. benchmarked face recognition on a **Jetson Nano**, achieving 98.642% precision with 165 ms single-identity latency (111 ms detection, 44 ms recognition, 10 ms matching) and a True Positive Rate (recall) of 97.9% at FAR=0.0001. Their multi-camera throughput analysis revealed that edge devices can serve 2 simultaneous camera feeds at 5 FPS for on-device processing, or 4–6 feeds in a hybrid edge-central architecture [25]. However, F1-score was not reported, limiting composite performance assessment.
- **Lightweight Recognition Networks:** George and Marcel proposed **EdgeFace**, a hybrid CNN-Transformer recognition model that achieves 99.73% accuracy on LFW [34] with only 1.77M parameters, ranking first among sub-2M-parameter models in the IJCB 2023 Efficient Face Recognition Competition and demonstrating that recognition (not just detection) can be made edge-viable [30].

2.2.4 Face Recognition Pipelines

The **MTCNN** framework [9] is widely cited for its effectiveness in joint face detection and alignment, using a cascaded structure where P-Net generates candidate windows, R-Net refines bounding boxes, and O-Net outputs final landmarks with facial point localization. For recognition, **FaceNet** [10] uses triplet loss to ensure that images of the same identity are closer in embedding space than images of different identities. More recent architectures include **RetinaFace** [35] for superior detection under heavy occlusion or extreme poses, and **ArcFace** [21], which has become the de facto standard for face embedding extraction due to its superior discriminative power through additive angular margin loss. Notably, the **MOS** framework [36] unifies face detection, landmark localization, and head pose estimation into a single lightweight model optimized for low-latency inference, making it particularly relevant for edge deployment where each pipeline stage must be computationally efficient.

2.2.5 Anomaly Detection Models

Current literature categorizes anomaly detection into **Reconstruction-based**, **Prediction-based**, and **Classification-based** methods. Reconstruction methods (often using GANs) learn “normal” patterns (e.g., enrolled individuals traversing campus) and flag deviations (e.g., unknown faces) based on high reconstruction errors. Prediction methods (using LSTMs) estimate future frames and flag anomalies when predictions diverge from reality [3]. Classification-based approaches directly model the boundary between normal and anomalous behavior. In the context of campus security monitoring, face recognition itself serves as a classification-based anomaly detector: an unrecognized face constitutes an anomaly triggering a high-risk alert to security personnel.

2.2.6 Robustness Techniques

To handle uncontrolled environments (e.g., low light at outdoor entry points or crowded indoor corridors), researchers emphasize preprocessing. Chitrapu et al. demonstrated that integrating **Contrast-Limited Adaptive Histogram Equalization (CLAHE)** and adaptive gamma correction significantly improves feature extraction in poor lighting [28].

2.2.7 Synthetic Data and Privacy

To address privacy concerns and data scarcity, recent literature advocates for synthetic datasets (e.g., VIGFace [37], Syn-Multi-PIE). These allow for training robust recognition models without using non-consensual images of real students, mitigating identity leakage risks while satisfying the large image count requirement for deep learning [38, 37]. This approach is particularly relevant for campus deployments where collecting large-scale, consented biometric datasets is logistically challenging.

2.3 Critique of Current Literature and Summary of Known Results

While deep learning models achieve high accuracy (e.g., 99.86% with enhanced FaceNet on LFW [1]—a near-saturated benchmark where marginal gains are diminishing), a significant gap remains between algorithmic performance and hardware feasibility. Many studies focus on accuracy on benchmark datasets (like LFW [34] or CASIA-WebFace [39]) but neglect the **hardware-software optimization** required for edge deployment [3]. Furthermore, while smart retrofitting using devices like Raspberry Pi has been explored [40], the specific architectural strategies to balance the “deployment contradiction” (high model complexity vs. low edge resources) in security contexts remain under-researched [15]. Most existing anomaly detection systems operate either fully on the cloud (high latency) or fully on the edge (low accuracy), necessitating a hybrid approach.

Table 2.2 summarizes five foundational works that collectively define the frontier of YOLO-based face detection and edge deployment. A critical analysis reveals four specific gaps that no single existing work addresses:

1. **No face-specific YOLO architectural modification has been tested on YOLOv11/v12.** Yu et al.’s YOLO-FaceV2 modifications (SEAM, RFE, NWD) were applied to YOLOv5 [13]; their transferability to newer architectures with attention mechanisms (e.g., YOLOv12) is untested.
2. **No study combines architectural optimization with INT8 quantization for face detection.** Choi et al. applied INT8 quantization only to the DnCNN preprocessing module while running YOLOv11 in FP32 [23]; the interaction between face-specific architectural changes and INT8 quantization of the detector itself is unexplored.
3. **No cross-platform edge benchmarking exists for face recognition pipelines.** Existing works benchmark on a single platform—Jetson AGX Orin [23], Jetson Nano [25], or mobile/edge devices [30]—but no study compares GPU-accelerated (Jetson), x86 CPU+iGPU (Intel N100), and ARM CPU (Raspberry Pi) platforms under identical conditions.
4. **Intel N-series processors are entirely absent from the face recognition literature.** Despite their low cost (\$100–200 for a complete system), availability, and OpenVINO support with VNNI acceleration, a thorough search of IEEE Xplore, Scopus, and arXiv yielded no published work benchmarking any YOLO model on Intel N100 or similar Alder Lake-N processors.

Table 2.1 provides a capability comparison across key dimensions, showing that no existing work covers all six criteria that this study addresses. Figure 2.3 provides a visual mapping of how each foundational work’s limitations define specific research gaps, and how this study addresses all four.

Table 2.1 compares existing works against this study across six dimensions: face-specific YOLO architectural modifications, INT8 quantization, cross-platform benchmarking, end-to-end detection-to-recognition pipeline, edge deployment, and Intel N100 support. While each prior work addresses one or two of these dimensions, no single existing study covers all six. This study is the first to combine all four architectural modifications with quantization-aware deployment across three distinct edge platforms.

In Figure 2.3, dashed arrows show how limitations of existing works—Yu & Zhang [13], Choi et al. [23], Ozen et al. [27], Dang et al. [25], George & Marcel [30], Tang [17], Aymon et al. [11], and Aboluhom & Kandilli [26]—map to four identified research gaps (centre, red). Bold arrows show how this study (bottom, green) addresses each

Table 2.1: Capability comparison of this study vs. prior work across six key dimensions.

Dimension	<i>Yu & Zhang '22</i>	<i>Choi et al. '25</i>	<i>Ozen et al. '24</i>	<i>Dang et al. '24</i>	<i>George & Marcel '23</i>	<i>Aboluhom & K. '25</i>	<i>This Study</i>
Face-Specific YOLO Mods	✓	—	—	—	—	—	✓
INT8 Quantization	—	✓	—	—	—	—	✓
Cross-Platform Benchmark	—	—	—	—	—	—	✓
End-to-End Pipeline	—	—	✓	?	—	✓	✓
Edge Deployed	—	✓	✓	✓	✓	✓	✓
Intel N100 Support	—	—	—	—	—	—	✓

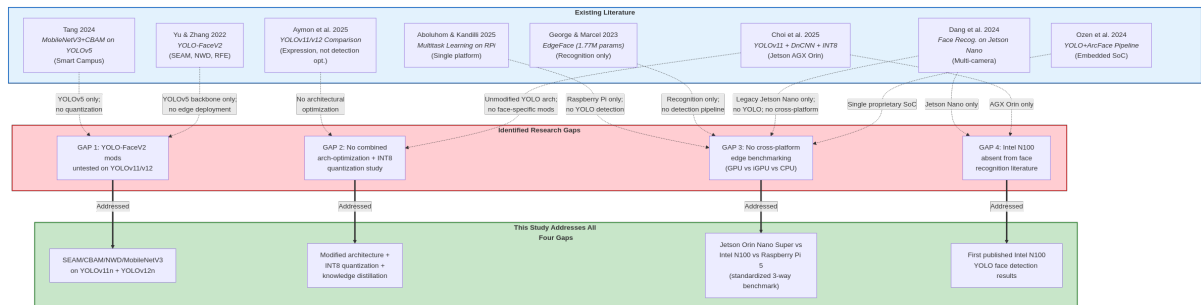


Figure 2.3: Literature gap analysis matrix mapping existing works to identified research gaps.

gap through YOLO-FaceV2 modifications on YOLOv11n/v12n, combined architectural optimization with INT8 quantization, three-way cross-platform benchmarking, and the first Intel N100 YOLO face detection results.

2.4 Contribution This Study Will Make to the Literature

This study will introduce a **unified hierarchical framework** that explicitly integrates **YOLO-based architectural optimization** (backbone replacement with MobileNetV3 [14], CBAM attention [20], SEAM module, NWD loss [33]) with **model compression** (INT8 quantization, knowledge distillation) and **collaborative edge-cloud inference** for the specific domain of campus security. Unlike general surveys, it will empirically benchmark optimized YOLO face detection and ArcFace [21] recognition pipelines across three distinct edge platforms—NVIDIA Jetson Orin Nano Super, Intel N100 mini PC, and Raspberry Pi 5—providing the first cross-platform latency–accuracy analysis on affordable hardware for campus deployment, with particular relevance to institutions in Sub-Saharan Africa and comparable resource-constrained environments [26].

Table 2.2: Comparison of Foundational Works in YOLO-Based Face Detection and Edge Deployment

Authors & Year	Methodology	Key Findings	Limitations
Yu & Zhang, 2022 [13]	YOLO-FaceV2: RFE module, SEAM attention, NWD loss [33], Repulsion Loss on YOLOv5	State-of-the-art on WIDER FACE [22]; SEAM handles occlusion; NWD improves small-face AP	Built on YOLOv5 only; no edge benchmarking; no quantization analysis
Choi et al., 2025 [23]	FP32 YOLOv11 + INT8-quantized DnCNN denoising on Jetson AGX Orin	INT8 DnCNN achieves near-FP16 mAP (<1% degradation); 31.6% throughput improvement (7.66 FPS combined)	No architectural modification to YOLO itself; single platform (AGX Orin); no recognition pipeline
Ozen et al., 2024 [27]	YOLOv3 + ArcFace [21] full pipeline on embedded camera SoC (ARM Cortex-A7 + NNIE, 1.0 TOPS): detect → landmark → quality → recognize	End-to-end pipeline at 12.5 FPS (80 ms total); per-stage optimization critical	Specific SoC not widely available; no quantization; no cross-platform comparison
Dang et al., 2024 [25]	Face recognition on Jetson Nano via Retina mnet + ViT-T; multi-camera throughput analysis with DeepStream	98.642% precision; 97.9% TPR; 165 ms single-ID latency; 2–6 cameras depending on architecture	Jetson Nano only (legacy, discontinued); no YOLO-based detection; TensorRT quantization (precision unspecified)
George & Marcel, 2023 [30]	EdgeFace: hybrid CNN-Transformer + Low Rank Linear layer for recognition	99.73% on LFW [34]; 1.77M params; 1st in IJCB 2023 sub-2M competition	Recognition only (no detection); no campus deployment context; 8-bit quantized variants submitted to competition but not primary focus

Chapter 3

Materials and Methods

3.1 Research Design

This research employs a **design science methodology** involving the design, implementation, and quantitative evaluation of a prototype security system. The approach follows an iterative build-and-evaluate cycle: architectural modifications are incrementally applied and their effects empirically measured through controlled experiments. The independent variables are the YOLO architecture variant (YOLOv11n, YOLOv12n), backbone type (default vs. MobileNetV3), attention module (none, CBAM, SEAM), loss function (CIoU vs. CIoU+NWD), quantization level (FP32, FP16, INT8), and target platform (Jetson, N100, RPi 5). The dependent variables are inference latency (ms), detection accuracy (mAP@0.5, mAP@0.5:0.95), recognition accuracy (FAR, FRR), throughput (FPS), model size (MB), and energy consumption (J/frame).

The system utilizes a **hierarchical computing structure**:

1. **Device Layer (Camera/Sensors):** Data capture via IP cameras at campus entry points and high-traffic areas.
2. **Edge Layer (Jetson/N100/Raspberry Pi):** Preprocessing, Face Detection (YOLO), Alignment, and lightweight Feature Extraction (ArcFace [21]).
3. **Cloud Layer (Server):** Database matching for large populations and long-term logging.

3.2 Specific Procedures

3.2.1 Image Acquisition

Video feeds are captured via IP cameras (e.g., Hikvision) positioned at campus entry points and high-traffic zones (e.g., main gates, lecture hall corridors, student congregation areas) [29]. Frames are streamed to the edge device via RTSP at 1080p/30 FPS and downsampled to the model's input resolution during preprocessing.

3.2.2 Model Selection and Architecture Modification

Two base architectures are investigated: **YOLOv11n** for its precision in noisy environments [11] and **YOLOv12n** as the latest attention-centric variant. Both are implemented using the Ultralytics framework [31]. Following the YOLO-FaceV2 paradigm [13], face-specific architectural modifications are applied:

- (i) Replacement of the default backbone with **MobileNetV3-Small** [14] for reduced parameter count [17]. Note: YOLOv11n uses a C3k2-based backbone (not the CSP-Darknet of earlier YOLO versions such as YOLOv4/v5); the replacement targets equivalent backbone layers while preserving the neck and head structure.
- (ii) Insertion of **CBAM** (Convolutional Block Attention Module) [20] attention into the neck for enhanced feature selection under occlusion.
- (iii) Addition of a **SEAM** (Scale-Equalizing Anchor Matching) module before detection heads to improve small-face recall [13].
- (iv) Adoption of **NWD** (Normalized Wasserstein Distance) loss [33] for robust tiny-face localization on the stride-8 (80×80) detection head.

These modifications target a $>30\%$ parameter reduction relative to the unmodified YOLOv11n baseline (approximately 2.6M parameters, 6.5 GFLOPs at 640×640 input) while maintaining $>90\%$ mAP@0.5 on the WIDER FACE [22] validation set. The mathematical foundations of each modification are detailed in Section 3.2.4. The proposed modified architecture is illustrated in Figure 3.1.

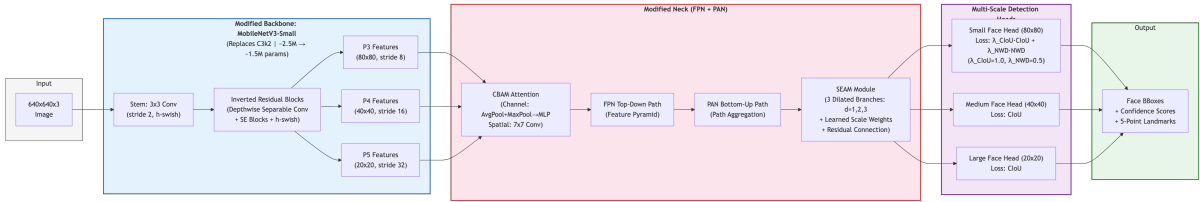


Figure 3.1: Proposed modified YOLO architecture for face detection.

The architecture in Figure 3.1 extends the YOLO-FaceV2 paradigm [13] from YOLOv5 to YOLOv11n/v12n. The standard C3k2-based backbone is replaced with MobileNetV3-Small [14] (blue), which uses depthwise separable convolutions and Squeeze-and-Excitation blocks to reduce parameters from approximately 2.5M to 1.5M. The neck (red) incorporates CBAM attention [20] for channel and spatial feature refinement and a SEAM module [13] with three dilated branches ($d = 1, 2, 3$) for occlusion-aware multi-scale processing. Multi-scale detection heads (purple) output face bounding boxes, with NWD loss [33] applied alongside CIoU loss ($\lambda_{CIoU} = 1.0$, $\lambda_{NWD} = 0.5$) on the small-face head (80×80).

3.2.3 Optimization

Post-training quantization (PTQ) using the QDQ (Quantize-DeQuantize) approach converts weights and activations to INT8, reducing memory footprint by approximately $4\times$ for edge deployment. While Choi et al. [23] applied INT8 quantization only to the DnCNN preprocessing module (keeping YOLOv11 in FP32), this study applies INT8 quantization directly to the YOLO detection model itself to maximize inference speed on edge hardware. Platform-specific export is performed:

- **TensorRT INT8 calibration** for Jetson Orin Nano Super, using 1,000 representative calibration images from the WIDER FACE training set.
- **OpenVINO INT8 via NNCF** (Neural Network Compression Framework [41]) for Intel N100, which replaces the deprecated Post-Training Optimization Toolkit (POT).
- **NCNN INT8/FP32** for Raspberry Pi 5 [24]. Note: the Cortex-A76 CPU in the RPi 5 does not provide hardware FP16 acceleration for neural network inference; NCNN will therefore use INT8 quantization where accuracy permits, with FP32 as the fallback.

Knowledge distillation will additionally be applied to transfer accuracy from a heavier YOLOv11l teacher model to the modified nano student, following the feature-based distillation paradigm [15]. The full four-phase optimization pipeline is shown in Figure 3.2.

Figure 3.2 details the four optimization phases. Phase 1 (blue) applies architectural modifications: MobileNetV3 backbone replacement [14] for an approximately $8-9\times$ FLOP reduction, CBAM attention [20] with reduction ratio $r = 16$, SEAM module [13] with three dilated branches, and NWD loss [33] on the stride-8 head. Phase 2 (orange) performs knowledge distillation [42] from the YOLOv11l teacher ($\sim 25\text{M}$ parameters) to the modified nano student ($\sim 1.8\text{M}$ parameters) using temperature $T = 4$, $\alpha = 0.5$, and $\beta = 0.1$. Phase 3 (red) applies QDQ INT8 post-training quantization with 1,000-image calibration [23], yielding an approximately $4\times$ size reduction. Phase 4 (green) exports to platform-specific runtimes: TensorRT for Jetson, OpenVINO via NNCF [41] for Intel N100, and NCNN [24] for Raspberry Pi 5.

3.2.4 Mathematical Background

This section formalizes the mathematical foundations of the four architectural modifications and the training methodology used in this study. These formulations underpin the implementation described in Sections 3 and provide the theoretical basis for the design choices.

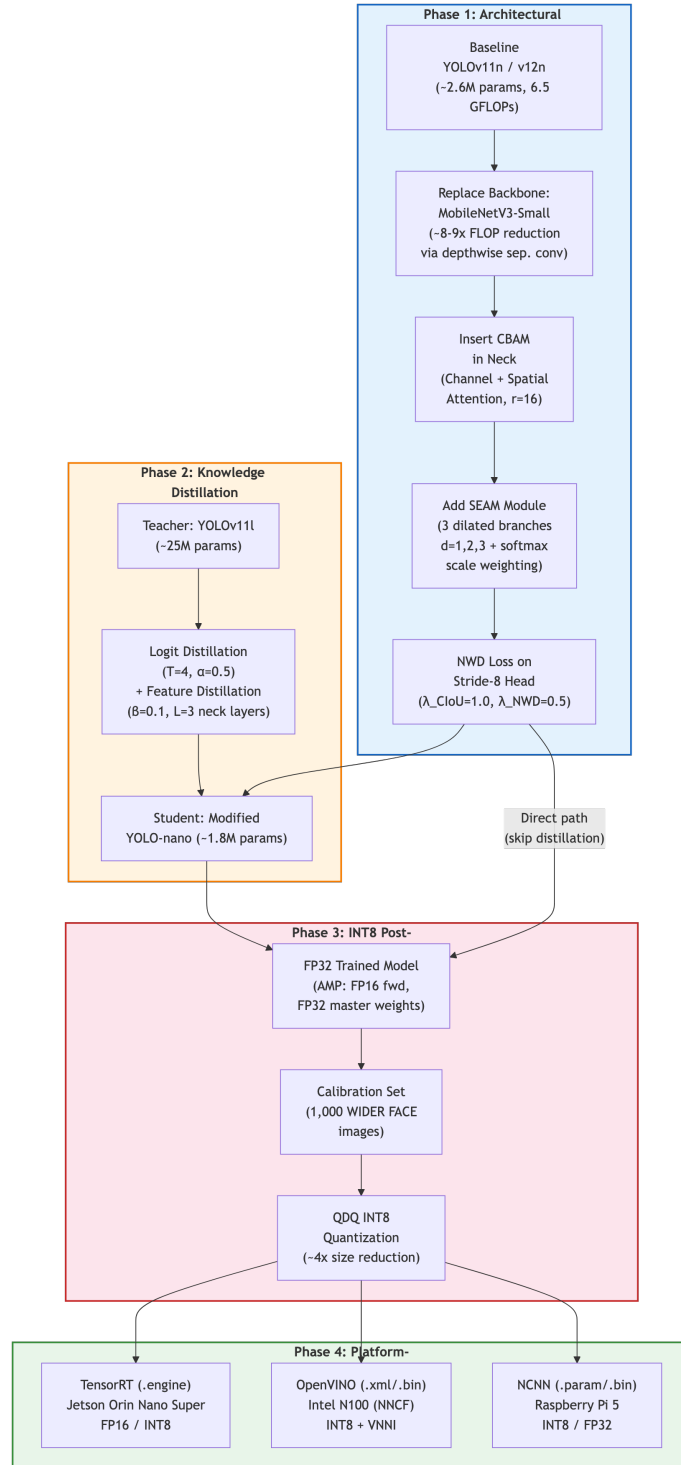


Figure 3.2: Four-phase model optimization strategy.

3.2.4.1 Depthwise Separable Convolutions (MobileNetV3 Backbone)

The computational advantage of replacing the default backbone with MobileNetV3-Small [14] stems from **depthwise separable convolutions** [43]. A standard convolution with kernel size $K \times K$, C_{in} input channels, and C_{out} output channels on a spatial feature map of size $H \times W$ requires:

$$\mathcal{C}_{\text{std}} = K^2 \cdot C_{\text{in}} \cdot C_{\text{out}} \cdot H \cdot W \quad (3.1)$$

floating-point operations. A depthwise separable convolution decomposes this into a *depthwise* convolution (one $K \times K$ filter per input channel) followed by a *pointwise* 1×1 convolution:

$$\mathcal{C}_{\text{dw}} = K^2 \cdot C_{\text{in}} \cdot H \cdot W + C_{\text{in}} \cdot C_{\text{out}} \cdot H \cdot W \quad (3.2)$$

The computational reduction ratio is:

$$\frac{\mathcal{C}_{\text{dw}}}{\mathcal{C}_{\text{std}}} = \frac{1}{C_{\text{out}}} + \frac{1}{K^2} \quad (3.3)$$

For a typical 3×3 kernel, this yields an approximately 8–9 $\times$ reduction in FLOPs, which is the primary mechanism by which MobileNetV3-Small achieves lower computational cost ($\sim 2.5\text{M}$ parameters) compared to the C3k2-based backbone of YOLOv11n ($\sim 3.2\text{M}$ backbone parameters).

MobileNetV3-Small further incorporates **Squeeze-and-Excitation (SE)** blocks [44] and the hardware-efficient **h-swish** activation $\sigma_h(x) = x \cdot \text{ReLU6}(x + 3)/6$, which approximates the swish function without requiring a full sigmoid computation. The SE block recalibrates channel-wise feature responses via global average pooling followed by a bottleneck MLP:

$$\mathbf{s} = \sigma \left(\mathbf{W}_2 \delta \left(\mathbf{W}_1 \frac{1}{HW} \sum_{i,j} \mathbf{x}_{:,i,j} \right) \right), \quad \hat{\mathbf{x}} = \mathbf{s} \odot \mathbf{x} \quad (3.4)$$

where σ is the sigmoid function, δ is ReLU, $\mathbf{W}_1 \in \mathbb{R}^{C/r \times C}$ and $\mathbf{W}_2 \in \mathbb{R}^{C \times C/r}$ are learned projection matrices with reduction ratio r , and $\odot$ denotes channel-wise multiplication.

3.2.4.2 CBAM: Convolutional Block Attention Module

CBAM [20] sequentially applies **channel attention** and **spatial attention** to refine feature maps in the YOLO neck. Given an intermediate feature map $\mathbf{F} \in \mathbb{R}^{C \times H \times W}$:

Channel Attention. The channel attention map $\mathbf{M}_c \in \mathbb{R}^{C \times 1 \times 1}$ is computed by aggregating spatial information via both global average pooling and global max pooling, passing each through a shared two-layer MLP (with reduction ratio $r = 16$), and com-

binning via element-wise summation:

$$\mathbf{M}_c(\mathbf{F}) = \sigma(\text{MLP}(\text{AvgPool}(\mathbf{F})) + \text{MLP}(\text{MaxPool}(\mathbf{F}))) \quad (3.5)$$

where $\text{MLP}(\mathbf{z}) = \mathbf{W}_2 \delta(\mathbf{W}_1 \mathbf{z})$ with $\mathbf{W}_1 \in \mathbb{R}^{C/r \times C}$ and $\mathbf{W}_2 \in \mathbb{R}^{C \times C/r}$.

Spatial Attention. After applying channel attention ($\mathbf{F}' = \mathbf{M}_c \odot \mathbf{F}$), the spatial attention map $\mathbf{M}_s \in \mathbb{R}^{1 \times H \times W}$ is computed by concatenating channel-wise average and max features and applying a 7×7 convolution:

$$\mathbf{M}_s(\mathbf{F}') = \sigma(f^{7 \times 7}([\text{AvgPool}_c(\mathbf{F}'); \text{MaxPool}_c(\mathbf{F}')])) \quad (3.6)$$

where $[\cdot; \cdot]$ denotes concatenation along the channel dimension. The refined output is $\hat{\mathbf{F}} = \mathbf{M}_s \odot \mathbf{F}'$. CBAM adds negligible parameters (a shared MLP and a single 7×7 convolution per insertion point) while providing both “what” (channel) and “where” (spatial) attention, which is particularly beneficial for selecting discriminative face features under occlusion [17]. The internal architecture of CBAM is illustrated in Figure 3.3.

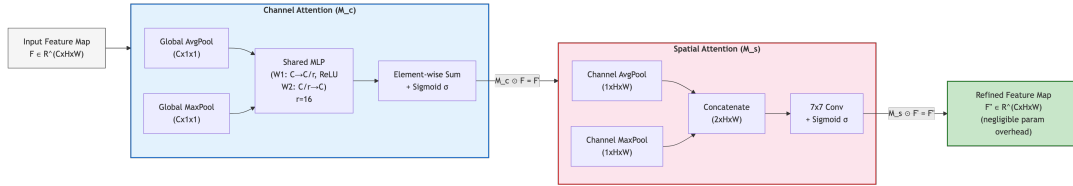


Figure 3.3: Internal architecture of the Convolutional Block Attention Module (CBAM) [20].

As shown in Figure 3.3, CBAM applies two sequential attention stages. Channel attention (left) aggregates spatial information via both global average pooling and global max pooling, passing each through a shared two-layer MLP with reduction ratio $r = 16$, and combining via element-wise summation followed by sigmoid activation. Spatial attention (right) concatenates channel-wise average and max pooled features and applies a 7×7 convolution with sigmoid activation. The sequential application refines “what” (channel) and “where” (spatial) to attend in the feature map, adding negligible parameter overhead per insertion point.

3.2.4.3 SEAM: Scale-Equalizing Anchor Matching

The SEAM module [13] addresses the well-known scale imbalance problem in face detection, where large faces dominate the training gradient and small faces are underrepresented. SEAM processes the input feature map $\mathbf{F} \in \mathbb{R}^{C \times H \times W}$ through $B = 3$ parallel branches, each with a different dilation rate $d_i = i$ (for $i = 1, 2, 3$), providing progressively

larger effective receptive fields:

$$\mathbf{F}_i = \text{SiLU}(\text{BN}(\text{Conv}_{3 \times 3}^{d=i}(\mathbf{F}))), \quad i = 1, \dots, B \quad (3.7)$$

A learned scale-aware weighting mechanism computes soft attention weights $\mathbf{w} = [w_1, \dots, w_B]$ via global average pooling followed by a linear layer and softmax:

$$\mathbf{w} = \text{Softmax}(\mathbf{W}_s \cdot \text{GAP}(\mathbf{F})), \quad \mathbf{W}_s \in \mathbb{R}^{B \times C} \quad (3.8)$$

The output is a weighted sum of branch features with a residual connection and a 1×1 fusion convolution:

$$\hat{\mathbf{F}} = \text{SiLU} \left(\text{BN} \left(\text{Conv}_{1 \times 1} \left(\sum_{i=1}^B w_i \cdot \mathbf{F}_i \right) \right) \right) + \mathbf{F} \quad (3.9)$$

By learning to balance contributions across receptive field scales, SEAM equalizes gradient flow so that small faces receive proportionally stronger supervision, improving recall on the WIDER FACE Hard subset.

3.2.4.4 NWD: Normalized Wasserstein Distance Loss

Standard IoU-based bounding box regression losses (including CIoU) degrade for very small objects because minor pixel offsets cause disproportionately large IoU changes. The Normalized Wasserstein Distance (NWD) loss [33] addresses this by modeling each bounding box as a 2D Gaussian distribution and measuring the distance between distributions rather than geometric overlap.

A bounding box $\mathbf{b} = (c_x, c_y, w, h)$ is mapped to a Gaussian $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ with:

$$\boldsymbol{\mu} = \begin{pmatrix} c_x \\ c_y \end{pmatrix}, \quad \boldsymbol{\Sigma} = \begin{pmatrix} (w/2)^2 & 0 \\ 0 & (h/2)^2 \end{pmatrix} \quad (3.10)$$

For two such Gaussians $\mathcal{N}_1(\boldsymbol{\mu}_1, \boldsymbol{\Sigma}_1)$ and $\mathcal{N}_2(\boldsymbol{\mu}_2, \boldsymbol{\Sigma}_2)$ with diagonal covariance, the squared 2-Wasserstein distance [45] admits the closed-form solution:

$$W_2^2(\mathcal{N}_1, \mathcal{N}_2) = \|\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2\|^2 + \|\boldsymbol{\sigma}_1 - \boldsymbol{\sigma}_2\|_F^2 \quad (3.11)$$

where $\boldsymbol{\sigma}_i$ are the diagonal standard deviation vectors and $\|\cdot\|_F$ reduces to the L_2 norm for diagonal matrices. The NWD similarity metric normalizes this distance via an exponential mapping:

$$\text{NWD}(\mathbf{b}_1, \mathbf{b}_2) = \exp \left(-\frac{\sqrt{W_2^2}}{\tau} \right) \in [0, 1] \quad (3.12)$$

where τ is a temperature constant controlling sensitivity. The NWD loss is then $\mathcal{L}_{\text{NWD}} =$

1 – NWD. Unlike IoU, which is undefined for non-overlapping boxes and degenerate for tiny boxes, NWD provides smooth gradients regardless of box size. Following the YOLO-FaceV2 strategy [13], NWD is applied only to the highest-resolution detection head (stride 8, 80×80 feature map at 640×640 input), where tiny faces ($< 32 \times 32$ pixels) are detected. The combined regression loss for the small-face head is:

$$\mathcal{L}_{\text{reg}} = \lambda_{\text{CIoU}} \cdot \mathcal{L}_{\text{CIoU}} + \lambda_{\text{NWD}} \cdot \mathcal{L}_{\text{NWD}} \quad (3.13)$$

with $\lambda_{\text{CIoU}} = 1.0$ and $\lambda_{\text{NWD}} = 0.5$.

3.2.4.5 Knowledge Distillation

Knowledge distillation [42] transfers the learned representations of a high-capacity *teacher* model (YOLOv11l, $\sim 25\text{M}$ parameters) to the lightweight *student* (modified YOLOv11n, $\sim 1.8\text{M}$ parameters). The student is trained to match the teacher’s softened output distribution using a temperature-scaled softmax. For logits $\mathbf{z}^T$ (teacher) and $\mathbf{z}^S$ (student), the distillation loss is the KL divergence on softened distributions:

$$\mathcal{L}_{\text{KD}} = T^2 \cdot D_{\text{KL}} \left(\text{softmax} \left(\frac{\mathbf{z}^T}{T} \right) \parallel \text{softmax} \left(\frac{\mathbf{z}^S}{T} \right) \right) \quad (3.14)$$

where $T = 4$ is the temperature parameter that controls the entropy of the softened distributions. Higher temperatures produce softer probability distributions, revealing inter-class relationships (“dark knowledge”) that the student cannot learn from hard labels alone. The T^2 scaling factor ensures that the gradient magnitudes remain consistent as T varies.

Additionally, **feature-level distillation** aligns intermediate feature maps between teacher and student neck layers using MSE loss:

$$\mathcal{L}_{\text{feat}} = \frac{1}{L} \sum_{l=1}^L \|\mathbf{F}_l^S - \mathbf{F}_l^T\|_F^2 \quad (3.15)$$

where $L = 3$ is the number of matched neck layers. The total training loss combines the standard detection task loss $\mathcal{L}_{\text{task}}$ (CIoU + objectness + classification) with both distillation components:

$$\mathcal{L}_{\text{total}} = (1 - \alpha) \cdot \mathcal{L}_{\text{task}} + \alpha \cdot \mathcal{L}_{\text{KD}} + \beta \cdot \mathcal{L}_{\text{feat}} \quad (3.16)$$

with $\alpha = 0.5$ and $\beta = 0.1$. The complete knowledge distillation architecture, including the interaction between teacher and student networks and the three loss components, is depicted in Figure 3.4.

Figure 3.4 illustrates the distillation process. The teacher model (YOLOv11l, $\sim 25\text{M}$

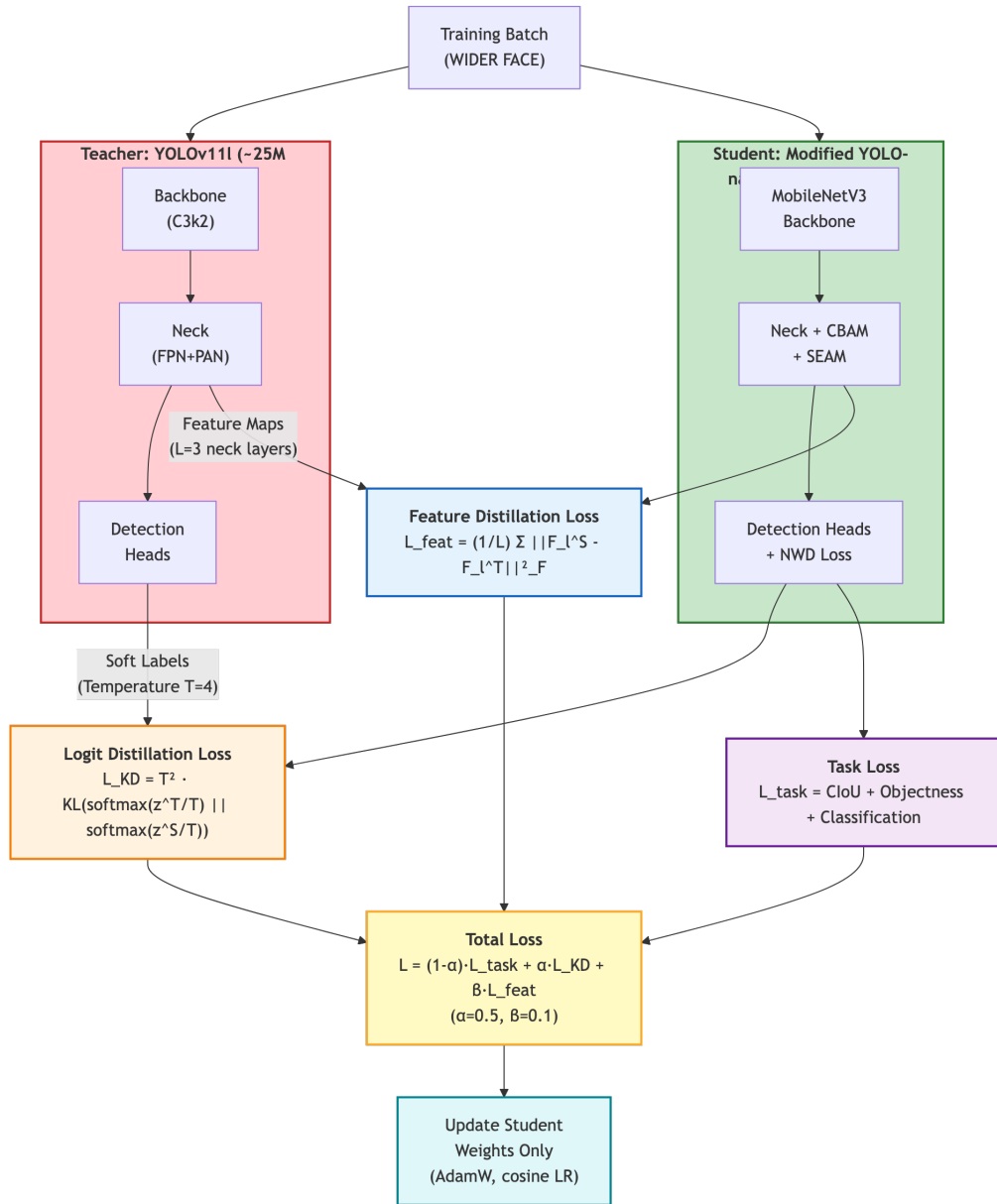


Figure 3.4: Knowledge distillation architecture showing teacher–student training.

parameters) provides soft labels via temperature-scaled softmax ($T = 4$) and intermediate feature maps from $L = 3$ neck layers. The student (modified YOLO-nano, $\sim 1.8\text{M}$ parameters) is trained with a combined loss comprising the standard task loss $\mathcal{L}_{\text{task}}$ (CIoU + objectness + classification), logit distillation $\mathcal{L}_{\text{KD}}$ (Eq. 3.14), and feature distillation $\mathcal{L}_{\text{feat}}$ (Eq. 3.15), weighted by $\alpha = 0.5$ and $\beta = 0.1$ respectively. Only the student weights are updated during training; the teacher remains frozen.

3.2.4.6 Training Optimization

All models are trained using the **AdamW** optimizer [46], which decouples weight decay from the gradient update to prevent the well-known interaction between L_2 regularization and adaptive learning rates:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta_t \left(\frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} + \lambda \boldsymbol{\theta}_t \right) \quad (3.17)$$

where $\hat{\mathbf{m}}_t$ and $\hat{\mathbf{v}}_t$ are bias-corrected first and second moment estimates, $\lambda = 5 \times 10^{-4}$ is the weight decay coefficient, and $\epsilon = 10^{-8}$.

The learning rate η_t follows a **cosine annealing** schedule [47] with linear warmup:

$$\eta_t = \begin{cases} \eta_0 \cdot t / t_{\text{warm}} & \text{if } t \leq t_{\text{warm}} \\ \eta_{\min} + \frac{1}{2}(\eta_0 - \eta_{\min}) \left(1 + \cos \left(\frac{\pi(t - t_{\text{warm}})}{t_{\max} - t_{\text{warm}}} \right) \right) & \text{otherwise} \end{cases} \quad (3.18)$$

with initial learning rate $\eta_0 = 10^{-3}$, minimum learning rate $\eta_{\min} = \eta_0 \cdot 0.01 = 10^{-5}$, warmup period $t_{\text{warm}} = 3$ epochs, and maximum epochs $t_{\max} = 300$. Training uses **automatic mixed precision (AMP)** to halve memory consumption and accelerate computation on GPU hardware by performing forward passes in FP16 while maintaining FP32 master weights for numerical stability.

3.2.5 Detection-to-Recognition Pipeline

Following the end-to-end YOLO→ArcFace paradigm of [27], the edge device captures video, applies **CLAHE** for lighting normalization [28], and runs the modified YOLO detector. Detected face crops are aligned using five-point landmark estimation [36] and passed through a lightweight **ArcFace** embedding network (MobileFaceNet backbone, producing 512-D embeddings) [21]. Resulting embeddings are matched locally via cosine similarity against a cached enrollment database of known individuals (students, staff, and authorized personnel). The matching threshold is determined via Receiver Operating Characteristic (ROC) analysis on the validation set, selecting the operating point that minimizes Equal Error Rate (EER). Faces whose similarity falls below the threshold are flagged as **high-risk** (unrecognized individuals), triggering an alert to security personnel

via the cloud layer for logging and secondary verification. The complete inference pipeline is depicted in Figure 3.5.

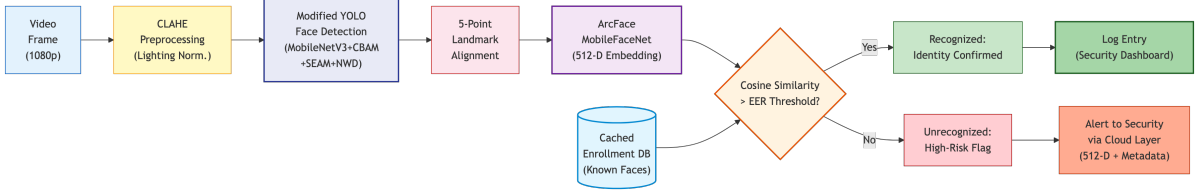


Figure 3.5: End-to-end detection-to-recognition inference pipeline.

The pipeline in Figure 3.5 follows the YOLO→ArcFace paradigm of Ozen et al. [27]. Each video frame undergoes CLAHE preprocessing for lighting normalization [28], modified YOLO face detection, five-point landmark alignment [36], and ArcFace embedding extraction producing a 512-dimensional vector [21]. Cosine similarity matching against the cached enrollment database classifies each detected face as *recognized* (enrolled individual) or *unrecognized* (potential high-risk individual). The matching threshold is set at the Equal Error Rate (EER) operating point determined via ROC analysis. Unrecognized faces are flagged as high-risk and escalated to security personnel via the cloud layer for verification.

3.3 Testing and Evaluation

3.3.1 Ablation Study

An ablation study isolates the contribution of each architectural modification. Five model variants are evaluated: (1) baseline YOLOv11n, (2) +MobileNetV3 backbone, (3) +CBAM attention, (4) +SEAM module, and (5) +NWD loss. Each variant is trained under identical conditions (same hyperparameters, data splits, random seed) and evaluated on the WIDER FACE [22] validation set.

3.3.2 Resolution Sensitivity

The system is tested on the **WIDER FACE** dataset [22] across input resolutions (160×160, 320×320, 640×640) to quantify the accuracy drop-off at lower resolutions typical of surveillance footage [16].

3.3.3 Environmental Stress Testing

Robustness is evaluated using high-density crowd scenarios inspired by CrowdFaceYOLO [18] and a custom test set containing disguised faces (masks, glasses, hats). The custom dataset comprises a minimum of 50 consented volunteers photographed under 4 conditions

(normal, mask, glasses, low-light) at 3 angles (frontal, 30°, 45°), yielding approximately 600 test images per condition.

3.3.4 Performance Metrics

The system is evaluated on:

- **mAP@0.5** and **mAP@0.5:0.95** — detection accuracy on WIDER FACE [22].
- **Inference Latency** — end-to-end time from frame input to threat assessment (targets: <30 ms on Jetson, <100 ms on N100 and RPi 5).
- **Recall** — sensitivity to small and occluded faces.
- **FPS** — throughput per platform under identical input conditions.
- **FAR/FRR** — False Acceptance Rate and False Rejection Rate for recognition reliability, following standard biometric evaluation methodology.

All metrics are reported per-platform to enable cross-platform comparison.

3.4 Research Population or Sample

3.4.1 Training Data

Detection models are trained on the **WIDER FACE** dataset [22] (32,203 images, 393,703 annotated faces). Recognition embeddings are trained on **CASIA-WebFace** [39] (494,414 images, 10,575 identities) and evaluated on **LFW** (Labeled Faces in the Wild) [34] (13,233 images, 5,749 identities).

3.4.2 Privacy-Preserving Training

To mitigate privacy risks and improve robustness against pose variations, **synthetic datasets** (e.g., **VIGFace** [37], **VariFace**) are utilized. These datasets generate virtual identities that mimic real-world distributions without exposing real student data [38].

3.4.3 Testing Data

A localized, consented dataset of volunteers (students) is created to simulate campus entry and high-traffic scenarios, including variations in lighting, accessories (masks/glasses), and camera angles. All participants sign informed consent forms approved by the institutional ethics review process.

3.5 Instrumentation (Pilot Study, Data Collection)

The study benchmarks three edge platforms spanning distinct compute tiers (GPU, iGPU, CPU-only), each paired with its most performant inference runtime:

Table 3.1: Target Edge Hardware Specifications

Platform	Compute	RAM	TDP	Cost	Runtime
Jetson Orin Nano Super	Ampere GPU (1024 CUDA)	8 GB	25 W	\$249	TensorRT
Intel N100 Mini PC	Intel UHD iGPU + VNNI	8–16 GB	6 W	\$100–200	OpenVINO
Raspberry Pi 5	Cortex-A76 CPU (4-core)	8 GB	12 W	\$80	NCNN

3.5.1 Hardware

Three edge platforms are selected to represent distinct resource tiers: the **NVIDIA Jetson Orin Nano Super** (GPU-accelerated edge AI), the **Intel N100 mini PC** (low-power x86 with integrated GPU and VNNI INT8 acceleration), and the **Raspberry Pi 5** (ARM CPU-only) [25, 40]. This three-way comparison provides the first published cross-platform YOLO face-detection benchmark on affordable edge hardware.

3.5.2 Software

The system is implemented in **Python 3.11+** using the **Ultralytics** framework [31] (YOLOv11/v12) with **PyTorch 2.1+** as the training backend. Models are exported to platform-optimized formats: **TensorRT** (FP16/INT8) for Jetson [23], **OpenVINO** (INT8 via NNCF [41]) for Intel N100, and **NCNN** [24] (INT8/FP32) for Raspberry Pi 5. **OpenCV 4.9+** is used for image preprocessing across all platforms. All experiments use a fixed random seed (42) for reproducibility.

3.5.3 Computational Budget and Resource Requirements

Training deep learning models for this study demands significant computational resources. This section details the hardware, time, and financial requirements for each experimental phase, justifying the budget necessary to reproduce the results.

3.5.3.1 Training Compute Requirements

Table 3.2 summarizes the computational requirements for each training phase. All estimates assume a single NVIDIA GPU with AMP (FP16 forward, FP32 master weights) enabled.

The GPU-hour estimates in Table 3.2 are based on the WIDER FACE training set (32,203 images, 640×640 input resolution) using an NVIDIA A100 40 GB or RTX 3090

Table 3.2: Training Compute Requirements per Experimental Phase

Phase	Epochs	Batch	VRAM	GPU-hrs	× Seeds
YOLOv11n main training	300	16	~8 GB	8–12	×5
YOLOv12n main training	300	16	~8 GB	8–12	×5
Ablation (5 variants)	100 each	16	~8 GB	15–20	×5
Knowledge distillation	200	8	~16 GB	10–15	×5
Total (single seed)	—	—	—	41–59	—
Total (×5 seeds)	—	—	—	205–295	—

24 GB class GPU. The ablation study trains five model variants (Table ??) for 100 epochs each under identical hyperparameters to isolate individual component contributions. The knowledge distillation phase requires approximately double the VRAM of standard training because both the teacher model (YOLOv11l, ~25M parameters) and the student model must reside in GPU memory simultaneously (see Equation 3.16).

The statistical validation protocol (Section 3.5.3; 5 independent runs per configuration with different random seeds) multiplies the base compute by a factor of 5, yielding a total requirement of approximately **200–300 GPU-hours**. This protocol is necessary to report means and standard deviations with sufficient statistical power for paired t-tests and McNemar tests at $\alpha = 0.05$ significance.

3.5.3.2 GPU Hardware Options

Table 3.3 compares viable GPU options for the training workload. The minimum VRAM requirement is 12 GB for standard training and 16–24 GB for knowledge distillation (teacher + student in memory at batch size 8).

Table 3.3: GPU Options for Training Workload

GPU	VRAM	Est. Cost	Supports KD?	Total Cost
Google Colab Pro+ (A100)	40 GB	\$49.99/mo [48]	Yes	\$100–200
Cloud A100 (on-demand)	40 GB	\$1.50–3.00/hr	Yes	\$300–900
NVIDIA RTX 3060 (local)	12 GB	\$300 (one-time)	Partial*	\$300+
NVIDIA RTX 4090 (local)	24 GB	\$1,600 (one-time)	Yes	\$1,600+

*RTX 3060 12 GB can run distillation at reduced batch size (4) with gradient accumulation, but training time approximately doubles.

For this study, training is conducted using cloud GPU instances (NVIDIA A100 40 GB) to accommodate the knowledge distillation phase and to enable parallel execution of ablation variants across multiple seeds. The estimated cloud compute budget is **\$300–600** for the complete training pipeline, depending on GPU availability and spot pricing.

3.5.3.3 Edge Hardware Budget

The three target edge platforms (Table 3.1) are required for the cross-platform benchmarking phase. Table 3.4 details the complete hardware acquisition budget.

Table 3.4: Edge Hardware Acquisition Budget

Item	Purpose	Cost (USD)
NVIDIA Jetson Orin Nano Super (8 GB)	GPU-accelerated edge inference	\$249
Intel N100 Mini PC (16 GB RAM)	x86 CPU+iGPU inference	\$100–200
Raspberry Pi 5 (8 GB)	ARM CPU-only inference	\$80
USB power meter (2×)	Energy measurement (N100, RPi 5)	\$30
microSD cards, cables, peripherals	Accessories	\$50
IP camera (Hikvision or equivalent)	Video capture for pilot study	\$50–100
Total edge hardware		\$559–709

3.5.3.4 Storage Requirements

The datasets and model artifacts require approximately 25 GB of storage:

- **WIDER FACE** (train + val + annotations): ~ 3.5 GB raw, ~ 5 GB processed (YOLO format + CLAHE)
- **CASIA-WebFace**: ~ 4 GB (494,414 images, 10,575 identities)
- **LFW**: ~ 200 MB (13,233 images)
- **Training checkpoints**: ~ 500 MB per run $\times$ (2 architectures + 5 ablation + 1 distillation) $\times$ 5 seeds ≈ 20 GB
- **Exported models**: ~ 50 MB per platform $\times$ 3 platforms $\times$ 3 precisions ≈ 450 MB

3.5.3.5 Total Budget Summary

Table 3.5 summarizes the complete financial budget for the study.

Table 3.5: Total Research Budget Summary

Category	Estimated Cost (USD)
Cloud GPU training (A100, 200–300 hrs)	\$300–600
Edge hardware acquisition	\$559–709
Cloud storage (datasets, checkpoints)	\$10–20
Miscellaneous (power, network, consumables)	\$50–100
Total	\$919–1,429

This budget is justified by the study’s contribution as the first cross-platform benchmark of face-specific YOLO modifications on affordable edge hardware. The edge hardware cost (\$559–709) is a one-time investment that directly supports the core research contribution (three-way platform comparison), while the cloud GPU cost (\$300–600) is necessary for the statistical rigor of 5-seed replication across 8 training configurations. The total budget of approximately \$1,000–1,400 is modest relative to comparable deep learning research, which typically requires \$5,000–50,000 in compute resources for large-scale vision experiments.

3.5.4 Pilot Study

A controlled test is conducted at a high-traffic entry point on the University of Buea campus (e.g., a main gate or lecture hall entrance). The system processes real-time video feeds on all three edge platforms in parallel, matching detected faces against an enrollment database of registered individuals (minimum 20 enrolled volunteers) and flagging unrecognized individuals as high-risk for security review over a 4-hour testing period. Metrics are collected in 30-minute intervals to assess temporal stability.

The cross-platform deployment and evaluation workflow is illustrated in Figure 3.6.

As depicted in Figure 3.6, the optimized FP32 model is exported via the Ultralytics engine to three platform-specific runtimes: TensorRT (FP16/INT8) for Jetson Orin Nano Super with CUDA acceleration, OpenVINO (INT8 via NNCF [41]) for Intel N100 with VNNI INT8 acceleration, and NCNN (INT8/FP32) for Raspberry Pi 5 using ARM NEON. Energy consumption is measured via on-board sensors (Jetson `tegrastats`) or external USB power meters (N100, RPi 5). All platforms are evaluated on identical test sets (WIDER FACE validation + campus dataset) using a standardized protocol ($N \geq 1,000$ inference runs, first 100 discarded as warm-up) and compared via Pareto-optimal accuracy–latency–cost analysis.

3.6 Treatment and Analysis of the Data

Data analysis focuses on key performance metrics defined for Edge AI [15]:

- **Latency:** The end-to-end time from frame capture to threat assessment (measured in milliseconds). Platform-specific targets derived from the hypothesis: **<30 ms** on Jetson Orin Nano Super (TensorRT), **<100 ms** on Intel N100 (OpenVINO), and **<100 ms** on Raspberry Pi 5 (NCNN) [25, 23]. Each measurement is repeated over $N \geq 1,000$ inference runs to obtain stable statistics; the first 100 runs are discarded as warm-up.
- **Accuracy:** Detection accuracy measured via **mAP@0.5** and **mAP@0.5:0.95** on WIDER FACE [22]. Recognition accuracy measured via **False Acceptance Rate**

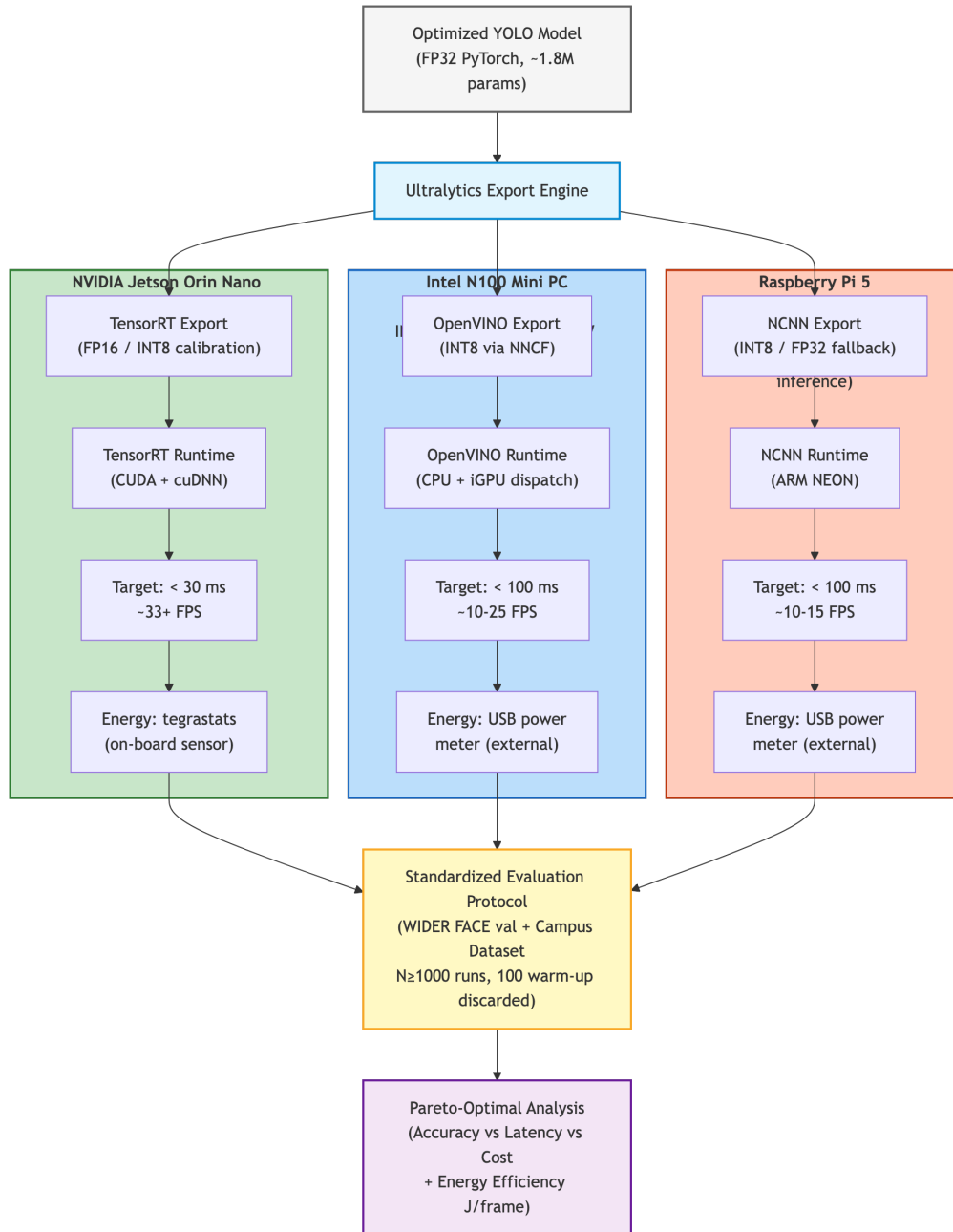


Figure 3.6: Cross-platform deployment and evaluation workflow.

(**FAR**) and **False Rejection Rate (FRR)** at the EER operating point, with a target of $>95\%$ recognition accuracy on the localized test set.

- **Throughput:** Frames Per Second (FPS) processed by each edge device, compared across all three platforms under identical input conditions [15].
- **Model Efficiency:** Parameter count, FLOPs, and model size (MB) before and after each architectural modification and quantization step, to isolate the contribution of each optimization.
- **Energy Efficiency:** Power draw (Watts) and energy consumed per inference (Joules/frame), measured via on-board sensors (Jetson `tegrastats`) or external USB power meters, critical for sustainable edge deployment [15].
- **Cross-Platform Comparison:** A standardized benchmark protocol evaluates all three platforms on identical test images (WIDER FACE validation set + localized campus dataset) using their respective optimized model exports. Results are presented as Pareto-optimal trade-off curves (accuracy vs. latency, accuracy vs. cost) to guide deployment decisions for resource-constrained institutions.

3.6.1 Statistical Analysis

Comparative analyses are performed between (i) the baseline YOLOv11n/v12n and the modified architectures (ablation study), (ii) the three edge platforms (cross-platform benchmark), and (iii) the “Cloud-Only” approach vs. the proposed “Edge-Cloud” architecture to quantify bandwidth savings ($>80\%$ target) and latency reduction. Statistical significance of accuracy differences between ablation variants is assessed using the **McNemar test** (for binary match/no-match outcomes) or **paired t-test** (for continuous latency measurements), with a significance level of $\alpha = 0.05$. Each experiment configuration is run 5 times with different random seeds to report mean and standard deviation.

The complete experimental protocol spanning training, optimization, benchmarking, and evaluation is summarized in Figure 3.7.

Figure 3.7 summarizes the four-phase research protocol. Phase 1 trains modified YOLO variants on WIDER FACE [22], CASIA-WebFace [39], and synthetic data [37, 38], with a five-variant ablation study replicated across five random seeds ($\sim 200\text{--}300$ GPU-hours total). Phase 2 optimizes the best variant via knowledge distillation [42] and QDQ INT8 quantization [23], followed by platform-specific export. Phase 3 benchmarks across all three edge platforms under identical conditions. Phase 4 evaluates robustness under resolution sensitivity [16], environmental stress (crowds, masks, low light) [18], multi-camera scalability, and a four-hour pilot study. Five categories of metrics are collected, with statistical significance assessed via McNemar tests and paired t-tests ($\alpha = 0.05$).

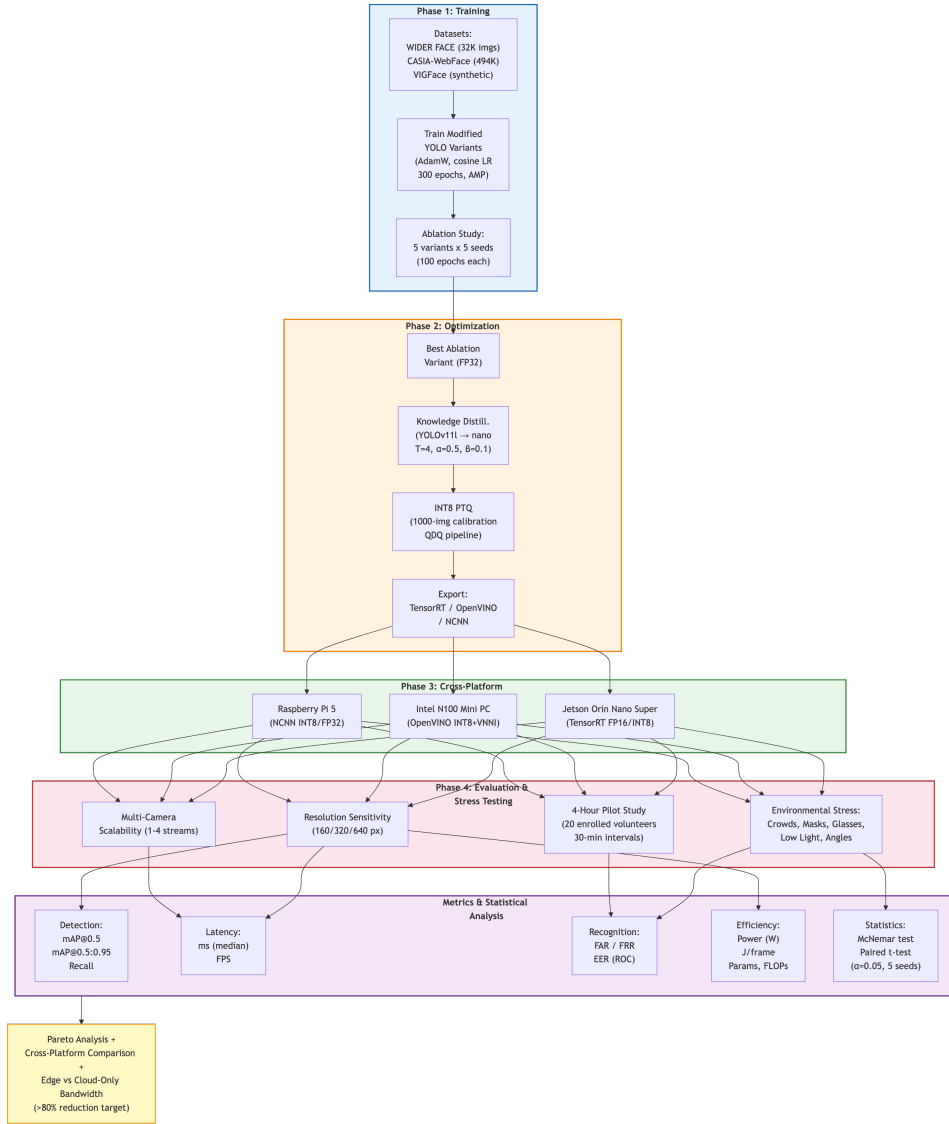


Figure 3.7: Complete experimental protocol spanning training, optimization, benchmarking, and evaluation.

3.7 Ethical Considerations

The study strictly adheres to **Privacy-by-Design** principles. By processing data at the edge, raw video footage of students is *not* transmitted to the cloud; only mathematical embeddings or anonymized logs are sent [15]. Furthermore, the use of **synthetic data** for model training prevents identity leakage from real-world training datasets, ensuring that the system is not trained on non-consensual images scraped from the internet [38, 37]. All real-world testing data is collected with explicit informed consent, and facial embeddings are stored in encrypted form on the edge device with access restricted to authorized personnel.

3.8 Scope of the Current Research

This research is delimited to the **technological architecture** of video-based security monitoring (specifically flagging unrecognized individuals as high-risk) across **busy university campus environments**, targeting entry points and areas with high student and staff populations. It focuses on the efficiency of **edge computing algorithms** (detection, recognition, optimization) and does not extend to broader behavioral analytics (e.g., detecting fights or weapons) or audio surveillance [3]. The system is designed to *alert* security personnel to potential threats rather than to autonomously grant or deny physical access. The study assumes a stable, albeit potentially low-bandwidth, connection between the edge and cloud layers for logging purposes. Hardware is evaluated as received from manufacturers without custom firmware or overclocking modifications.

Bibliography

- [1] M. M. Abid, T. Mahmood, R. Ashraf, C. M. N. Faisal, H. Ahmad, and A. A. Niaz, “Computationally intelligent real-time security surveillance system in the education sector using deep learning,” *PLOS ONE*, vol. 19, no. 7, p. e0301908, 2024.
- [2] M. S. Ejaz, S. Debnath, M. K. Hasan, and M. M. Alam, “Facial recognition-based entry system for student residence halls: Enhancing security and accessibility,” *Asian J. Res. Comput. Sci.*, vol. 16, no. 4, pp. 344–353, 2023.
- [3] D. R. Patrikar and M. R. Parate, “Anomaly detection using edge computing in video surveillance system: Review,” *Int. J. Multimed. Inf. Retr.*, vol. 11, no. 2, pp. 85–110, 2022.
- [4] M. Turk and A. Pentland, “Eigenfaces for recognition,” *J. Cogn. Neurosci.*, vol. 3, no. 1, pp. 71–86, 1991.
- [5] N. Dalal and B. Triggs, “Histograms of oriented gradients for human detection,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, vol. 1, 2005, pp. 886–893.
- [6] P. Viola and M. Jones, “Rapid object detection using a boosted cascade of simple features,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, vol. 1, 2001, pp. 511–518.
- [7] A. Aggarwal, M. Alshehri, M. Kumar, P. Sharma, O. Alfarraj, and V. Deep, “Principal component analysis, hidden markov model, and artificial neural network inspired techniques to recognize faces,” *Concurr. Comput. Pract. Exp.*, vol. 33, no. 9, p. e6157, 2021.
- [8] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet classification with deep convolutional neural networks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 25, 2012, pp. 1097–1105. [Online]. Available: <https://papers.nips.cc/paper/2012/hash/c399862d3b9d6b76c8436e924a68c45b-Abstract.html>
- [9] K. Zhang, Z. Zhang, Z. Li, and Y. Qiao, “Joint face detection and alignment using multi-task cascaded convolutional networks,” *IEEE Signal Process. Lett.*, vol. 23, no. 10, pp. 1499–1503, 2016.

- [10] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A unified embedding for face recognition and clustering,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 815–823.
- [11] U. Aymon, N. S. Kamarudin, and A. F. A. Nasir, “Facial expression recognition with yolov11 and yolov12: A comparative study,” 2025.
- [12] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.
- [13] J. Yu and Z. Zhang, “YOLO-FaceV2: A scale and occlusion aware face detector,” 2022.
- [14] A. Howard, M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan, Q. V. Le, and H. Adam, “Searching for MobileNetV3,” in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2019, pp. 1314–1324.
- [15] X. Wang, Q. Li, and W. Jia, “Cognitive edge computing: A comprehensive survey on optimizing large models and AI agents for pervasive deployment,” 2025.
- [16] A. C. Ömercikoğlu, M. M. Yönügül, and P. Erdoğan, “The impact of image resolution on face detection: A comparative analysis of MTCNN, YOLOv11 and YOLOv12 models,” 2025.
- [17] Z. Tang, “Research on face detection based on CBAM module and improved YOLOv5 algorithm in smart campus security,” in *Proc. 2024 Int. Conf. Artificial Intelligence and Future Education*. ACM, 2024, pp. 255–259.
- [18] W. Ahmad, A. Vashist, N. Sinha, M. Prasad, V. Shrivastava, and J. H. Muza-mal, “CrowdFaceYOLO: Advancing real-time face detection in high-density crowded area,” in *2024 Artificial Intelligence for Business (AIB)*, 2024, pp. 92–93.
- [19] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, “Edge computing: Vision and challenges,” *IEEE Internet Things J.*, vol. 3, no. 5, pp. 637–646, 2016.
- [20] S. Woo, J. Park, J.-Y. Lee, and I. S. Kweon, “CBAM: Convolutional block attention module,” in *Proc. European Conf. Computer Vision (ECCV)*, 2018, pp. 3–19.
- [21] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, “ArcFace: Additive angular margin loss for deep face recognition,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 4690–4699.

- [22] S. Yang, P. Luo, C.-C. Loy, and X. Tang, “WIDER FACE: A face detection benchmark,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 5525–5533.
- [23] S. Choi, J. Lee, and H. Kim, “YOLOv11-based masked face detection with DnCNN denoising and INT8 quantization on NVIDIA Jetson AGX Orin,” *Electronics*, vol. 15, no. 1, p. 143, 2025.
- [24] nihui, “NCNN: A high-performance neural network inference framework for mobile,” 2017, tencent. [Online]. Available: <https://github.com/Tencent/ncnn>
- [25] H. Dang, T. Nguyen, P. Le, and V. Tran, “Face recognition on Jetson Nano: Multi-camera throughput and precision analysis,” *J. Sci. Technol.*, vol. 34, no. 1, pp. 1–10, 2024.
- [26] A. A. A. Aboluhom and I. Kandilli, “Real-time facial recognition via multitask learning on Raspberry Pi,” *Sci. Rep.*, vol. 15, no. 1, p. 28467, 2025.
- [27] E. Ozen, F. Alim, S. B. Okcu, E. Kavakli, and C. Cigla, “Real-time face recognition system at the edge,” 04 2024.
- [28] P. Chitrapu, M. K. Morampudi, and H. K. Kalluri, “Robust face recognition using deep learning and ensemble classification,” *IEEE Access*, vol. 13, pp. 99 957–99 969, 2025.
- [29] A. S. Latef and M. Y. Kamil, “Facial recognition technology-based attendance management system application in smart classroom,” *Iraqi J. Comput. Sci. Math.*, pp. 136–158, 2023.
- [30] A. George and S. Marcel, “EdgeFace: Efficient face recognition model for edge devices,” 2023.
- [31] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics YOLO,” 2023, version 8.0+. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [32] H. M. Yisihak and L. Li, “Advanced face detection with YOLOv8: Implementation and integration into AI modules,” *OALib*, vol. 11, no. 11, pp. 1–19, 2024.
- [33] J. Wang, C. Xu, W. Yang, and L. Yu, “A normalized Gaussian Wasserstein distance for tiny object detection,” in *Proc. AAAI Conf. Artificial Intelligence*, 2021.
- [34] G. B. Huang, M. Ramesh, T. Berg, and E. Learned-Miller, “Labeled faces in the wild: A database for studying face recognition in unconstrained environments,” University of Massachusetts, Amherst, Tech. Rep. 07-49, 2007. [Online]. Available: <http://vis-www.cs.umass.edu/lfw/>

- [35] J. Deng, J. Guo, E. Ververas, I. Kotsia, and S. Zafeiriou, “RetinaFace: Single-shot multi-level face localisation in the wild,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 5203–5212.
- [36] Y. Liu, Z. Gu, S. Gao, D. Wang, Y. Zeng, and J. Cheng, “MOS: A low latency and lightweight framework for face detection, landmark localization, and head pose estimation,” 2021.
- [37] M. Kim, M.-C. Sagong, G. P. Nam, J. Cho, and I.-J. Kim, “VIGFace: Virtual identity generation for privacy-free face recognition,” 2024.
- [38] P. Borsukiewicz, F. Boutros, I. E. Olatunji, C. Beumier, W. C. Ouedraogo, J. Klein, and T. F. Bissyandé, “Beyond real faces: Synthetic datasets can achieve reliable recognition performance without privacy compromise,” 2025.
- [39] D. Yi, Z. Lei, S. Liao, and S. Z. Li, “Learning face representation from scratch,” 2014.
- [40] A. Generosi, T. Agostinelli, and M. Mengoni, “Smart retrofitting for human factors: A face recognition-based system proposal,” *Int. J. Interact. Des. Manuf. (IJIDeM)*, vol. 17, no. 1, pp. 421–433, 2023.
- [41] Intel Corporation, “Neural Network Compression Framework (NNCF) for OpenVINO,” 2023, replaces deprecated Post-Training Optimization Toolkit (POT). [Online]. Available: <https://github.com/openvinotoolkit/nncf>
- [42] G. Hinton, O. Vinyals, and J. Dean, “Distilling the knowledge in a neural network,” 2015.
- [43] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, “MobileNets: Efficient convolutional neural networks for mobile vision applications,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2017.
- [44] J. Hu, L. Shen, and G. Sun, “Squeeze-and-excitation networks,” in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 7132–7141.
- [45] D. C. Dowson and B. V. Landau, “The Fréchet distance between multivariate normal distributions,” *Journal of Multivariate Analysis*, vol. 12, no. 3, pp. 450–455, 1982.
- [46] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2019.
- [47] —, “SGDR: Stochastic gradient descent with warm restarts,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2017.

- [48] Google, “Google Colab pricing,” 2024, accessed: 2025-12-01. [Online]. Available: <https://colab.research.google.com/signup>

Appendix A

Supplementary Data

This appendix provides supplementary data referenced throughout the main chapters, including the complete model architecture specifications, full training hyperparameter tables, dataset statistics, and the YOLO-format model configuration used for Ultralytics integration.

A.1 Complete Training Hyperparameters

Table A.1 lists all hyperparameters used for the main training runs (YOLOv11n and YOLOv12n). These settings are identical across all ablation variants to ensure fair comparison.

A.2 Knowledge Distillation Hyperparameters

Table A.2 lists the hyperparameters specific to the knowledge distillation phase (Section ??).

A.3 Dataset Statistics

Table A.3 summarizes the datasets used in this study. Detection models are trained on WIDER FACE; recognition embeddings use CASIA-WebFace for training and LFW for evaluation.

The WIDER FACE dataset contains faces across three difficulty levels based on detection challenge: **Easy** (large, unoccluded, frontal faces), **Medium** (moderate occlusion and pose variation), and **Hard** (small, heavily occluded, or extreme-pose faces). The Hard subset is particularly relevant for evaluating the SEAM and NWD modifications, as it contains the small and occluded faces these modules are designed to address.

A.4 Modified YOLO Architecture Specification

The following tables specify the layer-by-layer architecture of the proposed modified YOLOv11n with MobileNetV3-Small backbone, CBAM attention, and SEAM module. This architecture is expressed as a YAML configuration for the Ultralytics frame-

Table A.1: Complete Training Hyperparameters

Category	Parameter	Value
Optimizer	Algorithm	AdamW
	Initial learning rate (η_0)	1×10^{-3}
	Final LR multiplier ($\eta_0 \cdot \text{lrf}$)	1×10^{-5}
	Momentum (β_1)	0.937
	Weight decay (λ)	5×10^{-4}
	Cosine annealing	Enabled
Schedule	Maximum epochs	300
	Warmup epochs	3
	Warmup momentum	0.8
	Warmup bias LR	0.1
	Early stopping patience	50 epochs
Training	Batch size	16
	Input resolution	640×640
	Automatic Mixed Precision	Enabled (FP16 forward, FP32 master weights)
Augmentation	HSV hue shift	0.015
	HSV saturation shift	0.7
	HSV value shift	0.4
	Translation	0.1
	Scale	0.5
	Horizontal flip	0.5
	Mosaic	1.0
	CLAHE (clip limit / grid)	2.0 / 8×8
Reproducibility	Random seed	42 (base); 5 seeds total
	Checkpoint interval	Every 10 epochs

Table A.2: Knowledge Distillation Hyperparameters

Parameter	Value
Teacher model	YOLOv11l (~ 25 M parameters)
Student model	Modified YOLOv11n (full proposed architecture)
Temperature (T)	4.0
Distillation weight (α)	0.5
Feature loss weight (β)	0.1
Feature matching layers	neck.0, neck.1, neck.2
Epochs	200
Batch size	8
Optimizer	AdamW
Initial learning rate	5×10^{-4}
Cosine annealing	Enabled
AMP	Enabled

Table A.3: Dataset Statistics

Dataset	Images	Faces/IDs	Size	Purpose
WIDER FACE (train)	12,880	159,424 faces	~1.7 GB	Detection training
WIDER FACE (val)	3,226	39,708 faces	~0.4 GB	Detection validation
WIDER FACE (test)	16,097	— (no labels)	~1.4 GB	Detection testing
CASIA-WebFace	494,414	10,575 IDs	~4.0 GB	Recognition training
LFW	13,233	5,749 IDs	~200 MB	Recognition evaluation
Campus test (custom)	~600	≥ 50 IDs	~100 MB	Pilot study evaluation

work and is generated programmatically by the `generate_model_yaml()` function in `src/models/face_detector.py`.

A.4.1 Backbone (MobileNetV3-Small)

Table A.4: Modified Backbone Architecture (MobileNetV3-Small)

Layer	Module	Output Ch.	Stride	Notes
0	MobileNetV3Stage1	64	8	C3 features (pretrained ImageNet)
1	MobileNetV3Stage2	128	16	C4 features
2	MobileNetV3Stage3	256	32	C5 features

Each stage wraps the corresponding layers of torchvision’s MobileNetV3-Small, using depthwise separable convolutions with SE blocks and h-swish activations. A 1×1 projection convolution at the end of each stage maps the native MobileNetV3 channel count to the target channel dimensions (64, 128, 256) for compatibility with the YOLO neck.

A.4.2 Head (FPN/PAN Neck + Detection)

The CBAM modules are inserted after the C3k2 blocks in the FPN pathway (layers 6 and 10). The SEAM module (layer 11) is placed before the P3 detection head to improve small-face recall at the highest-resolution feature map. NWD loss is applied at the loss computation level to the P3 head predictions (stride 8) and does not appear in the architecture table.

A.5 Quantization Calibration Configuration

Table A.6 details the INT8 calibration settings for each target platform.

All calibration images are drawn from the WIDER FACE validation set to avoid data leakage from the training partition. The ONNX intermediate format (opset 17, simplified, fixed input shape $1 \times 3 \times 640 \times 640$) serves as the common export path for all three platform-specific runtimes.

Table A.5: Modified Head Architecture (FPN/PAN Neck with CBAM and SEAM)

Layer	Module	Output Ch.	Notes
<i>FPN top-down pathway</i>			
3	Upsample ($2\times$)	256	Upsample C5
4	Concat	384	Fuse with C4 (layer 1)
5	C3k2 ($\times 2$)	128	Feature refinement
6	CBAM	128	Channel + spatial attention
7	Upsample ($2\times$)	128	Upsample P4
8	Concat	192	Fuse with C3 (layer 0)
9	C3k2 ($\times 2$)	64	Feature refinement
10	CBAM	64	Channel + spatial attention
11	SEAM	64	Scale-equalizing attention
<i>PAN bottom-up pathway</i>			
12	Conv $3 \times 3 / 2$	64	Downsample P3
13	Concat	192	Fuse with P4 (layer 6)
14	C3k2 ($\times 2$)	128	Feature refinement
15	Conv $3 \times 3 / 2$	128	Downsample P4
16	Concat	384	Fuse with C5 (layer 2)
17	C3k2 ($\times 2$)	256	Feature refinement
<i>Detection heads</i>			
18	Detect	—	Multi-scale: P3 (80×80), P4 (40×40), P5 (20×20)

Table A.6: INT8 Quantization Calibration Configuration per Platform

Platform	Runtime	Calib. Images	Method
Jetson Orin Nano Super	TensorRT	1,000	Entropy calibration (from WIDER FACE val)
Intel N100 Mini PC	OpenVINO (NNCF)	300	Mixed preset quantization
Raspberry Pi 5	NCNN	300	Quantization table generation

A.6 Benchmark Protocol Details

The cross-platform benchmark follows a standardized protocol to ensure fair comparison:

1. **Warmup:** 100 inference iterations are executed and discarded to allow runtime optimization (e.g., TensorRT kernel auto-tuning, CPU cache warming).
2. **Measurement:** 1,000 inference iterations are timed. Each iteration processes a single 640×640 image (batch size 1) to reflect real-time single-camera operation.
3. **Repetition:** The full measurement is repeated 5 times with different random seeds (42, 43, 44, 45, 46) to capture run-to-run variance.
4. **Statistics:** Mean, standard deviation, median (P50), 95th percentile (P95), and 99th percentile (P99) latency are reported.
5. **Power measurement:** Jetson power is measured via `tegrastats`; Intel N100 via RAPL energy counters; Raspberry Pi 5 via external USB power meter. Power is sampled at 10 Hz during the 1,000-iteration measurement window.
6. **Thermal control:** All platforms operate with stock cooling in a $25 \pm 2^\circ\text{C}$ ambient environment. A 60-second idle period precedes each measurement run to allow thermal stabilization.

A.7 Resolution Sweep Configuration

The resolution sensitivity study (Section ??) evaluates detection accuracy at three input resolutions to simulate varying surveillance camera quality:

Table A.7: Resolution Sweep Parameters

Resolution	Feature Map (P3)	Simulated Scenario
160×160	20×20	Low-resolution legacy cameras
320×320	40×40	Standard VGA-quality surveillance
640×640	80×80	HD surveillance (primary operating point)

A.8 Environmental Stress Test Conditions

Table A.8 details the conditions under which the custom campus test dataset is captured for the environmental stress evaluation (Section ??).

All images are captured at three camera heights (1.5 m, 2.0 m, 2.5 m) to simulate typical CCTV mounting positions at campus entry points and along high-traffic corridors. Each volunteer is photographed in all conditions during a single session, and images are captured in JPEG format at 1920×1080 resolution before preprocessing.

Table A.8: Environmental Stress Test Dataset Conditions

Condition	Description	Est. Images
Normal (frontal)	Unoccluded, frontal pose, normal lighting	~150
Surgical mask	Lower face occluded by medical/surgical mask	~150
Glasses	Clear or tinted eyeglasses/sunglasses	~100
Low-light	Indoor dim lighting or evening conditions	~100
30° angle	Head turned approximately 30° from frontal	~50
45° angle	Head turned approximately 45° from frontal	~50
Total	Per volunteer $\times \geq 50$ volunteers	~600

Appendix B

Research Instruments

This appendix contains the research instruments used for data collection during the pilot study, including the informed consent form, the volunteer enrollment protocol, and the system evaluation checklist.

B.1 Informed Consent Form

The following informed consent form is administered to all volunteers prior to enrollment in the pilot study. The form complies with institutional ethics review requirements.

INFORMED CONSENT FOR PARTICIPATION IN RESEARCH

*Real-Time Facial Recognition Deep Learning Architecture
for Secure Campus Environments*

Principal Investigator: [Name]

Institution: University of Buea

Contact: [Email / Phone]

Purpose of the Study:

You are invited to participate in a research study evaluating a facial recognition system for campus security. The study aims to assess the accuracy and speed of an automated face detection and recognition system deployed at campus entry points and high-traffic areas to identify high-risk individuals.

What You Will Be Asked to Do:

1. Have your photograph taken under multiple conditions: normal (frontal), wearing a surgical mask, wearing glasses, in low-light conditions, and at 30° and 45° head angles.
2. Walk through the monitored entry point during the 4-hour pilot test period (at least 3 entries per volunteer).
3. Allow the system to capture video frames of your face during entry events.

Data Handling and Privacy:

- Your facial images will be processed *locally* on the edge device. **No raw images are transmitted to external servers.**
- Only mathematical feature vectors (512-dimensional numerical embeddings) are stored; these cannot be reverse-engineered into facial images.
- All stored embeddings are encrypted (AES-256) and access-restricted to the research team.
- Your data will be assigned an anonymous identifier (e.g., V001) and will not be linked to your name in any publication.
- You may withdraw from the study at any time, and your data will be permanently deleted within 48 hours of your withdrawal request.
- All data will be securely deleted within 6 months of the study’s completion.

Risks:

The risks of participation are minimal. Your photographs will be used exclusively for this research and will not be shared externally. The system does *not* grant or deny physical access to any facility.

Benefits:

There are no direct benefits to you. Your participation will contribute to research on affordable campus security solutions for institutions in resource-constrained environments.

Consent:

I have read and understood the above information. I voluntarily agree to participate in this study.

Participant Name (print)

Signature

Date

Researcher Signature

B.2 Volunteer Enrollment Protocol

The following protocol is followed by the research team to enroll each volunteer into the facial recognition system’s database.

1. **Consent verification:** Confirm that the volunteer has signed the informed consent form (Appendix B.1).
2. **Assign anonymous ID:** Assign the volunteer a sequential identifier (V001, V002, ...).
3. **Capture enrollment images:**
 - (a) Position the volunteer 1.0–1.5 m from the camera at eye level.
 - (b) Capture 5 frontal images under normal indoor lighting.
 - (c) Capture 3 images with slight head rotations ($\pm 15^\circ$).
 - (d) Ensure all images are sharp and well-lit before proceeding.
4. **Extract embeddings:** Run the ArcFace embedding extractor on each enrollment image.
 - (a) Detect the face using the trained YOLO model.
 - (b) Align the detected face crop using five-point landmarks.
 - (c) Resize to 112×112 pixels.
 - (d) Extract the 512-D embedding and L2-normalize.
5. **Compute enrollment template:** Average the L2-normalized embeddings across all enrollment images to produce a single enrollment template for the volunteer. Re-normalize the averaged embedding to unit length.
6. **Store template:** Save the enrollment template to the encrypted local database, indexed by the anonymous ID.
7. **Verification check:** Immediately test recognition by having the volunteer walk through the entry point once. Confirm that the system correctly identifies them with a cosine similarity score above the EER threshold.
8. **Log enrollment:** Record the enrollment timestamp, number of images captured, and verification result.

B.3 Pilot Study Data Collection Protocol

The following protocol governs data collection during the 4-hour pilot study at a University of Buea campus entry point.

B.3.1 Pre-Test Setup

1. Install and power on all three edge platforms (Jetson Orin Nano Super, Intel N100, Raspberry Pi 5) at the designated entry point.
2. Connect the IP camera (1080p, 30 FPS, RTSP stream) to the local network.
3. Verify that all three platforms receive the camera feed simultaneously.
4. Load the optimized INT8 model on each platform (TensorRT, OpenVINO, NCNN respectively).
5. Load the enrollment database (minimum 20 volunteers).
6. Start power monitoring tools: `tegrastats` (Jetson), RAPL logger (N100), USB meter logging (RPi 5).
7. Synchronize system clocks across all platforms using NTP.
8. Record ambient temperature and lighting conditions.

B.3.2 Test Execution

1. Run the inference pipeline continuously on all three platforms for 4 hours.
2. Record all detection and recognition events with timestamps, bounding box coordinates, cosine similarity scores, and platform identity.
3. At 30-minute intervals, record:
 - Ambient temperature and lighting conditions.
 - Platform CPU/GPU temperature.
 - Cumulative true positives (TP), false positives (FP), and false negatives (FN).
 - Mean inference latency over the preceding interval.
4. Volunteers are instructed to enter the monitored area at natural walking speed, without deliberately posing for the camera.
5. A research assistant manually logs each entry event (ground truth) on a separate device.

B.3.3 Post-Test Analysis

1. Compare system detections against the manual ground truth log to compute TP, FP, FN per platform.
2. Calculate FAR, FRR, and overall recognition accuracy per platform per 30-minute interval.
3. Compute latency statistics (mean, P50, P95, P99) from recorded timestamps.
4. Compute energy consumption (Joules/frame) from power monitoring logs.
5. Export all results to CSV for statistical analysis (paired t-tests, McNemar tests).

B.4 System Evaluation Checklist

The following checklist is used to verify that all experimental conditions are met before each benchmark or pilot test run.

Table B.1: Pre-Experiment Verification Checklist

#	Verification Item	Pass?
1	Model weights loaded and inference produces valid output	☐
2	Input resolution set to 640×640	☐
3	Correct quantization precision (FP32/FP16/INT8) confirmed	☐
4	Random seed set and recorded	☐
5	Test dataset is the designated split (no training data leakage)	☐
6	Power monitoring tool running and producing readings	☐
7	Ambient temperature within $25 \pm 2^\circ\text{C}$	☐
8	60-second thermal stabilization period completed	☐
9	System clocks synchronized (NTP)	☐
10	Enrollment database loaded with ≥ 20 volunteers	☐
11	Camera feed active and streaming at 1080p/30 FPS	☐
12	Disk space sufficient for logs (>1 GB free)	☐